v7 Exhaustive Technical Report

Reinforcement-Learning-Based Link Adaptation: Full System Design, Algorithm Choices, Source Code, and Results

RL-AMC Case Study — v7 Complete Technical Reference

August 8, 2026

Abstract

This is the complete, code-level technical reference for the v7 campaign of the OLLA-relative reinforcement-learning link-adaptation project. It documents, without abridgment, the full physical-layer and channel model, the CQI/BLER/HARQ physics, the Markov decision process formalization, all three learning algorithms (tabular Q-learning, tabular actor-critic, and deep A2C) with their complete source code, the full comparator suite, the entire seven-version design history from v1 through v7 (including every bug found and fixed at the code level), the pre-registration and experimental harness, all nine campaign points plus three diagnostic fix iterations plus the OLLA target-BLER sweep with exact numbers, and an appendix reproducing every legacy module reused from earlier versions. It is written as a standalone reference: a reader with no prior exposure to this project should be able to reconstruct the system from this document alone. It is a companion to, and supersedes in detail, the shorter `v7_technical_process_report.pdf` and `v7_comprehensive_evaluation_report.pdf`.

Contents

I Problem Statement and Physical-Layer System Model 4

1 The Link Adaptation Problem 4
- 1.1 Design Goals of the v7 Simulation . . . 4

2 Propagation and Channel Model 4
- 2.1 Link Budget . . . 4
- 2.2 Small-Scale (Multipath) Fading: Jakes Model per User . . . 4
- 2.3 Interference Bursts (Non-Stationarity) . . . 5
- 2.4 Full Channel Generator Source . . . 5

3 CQI Feedback Model 7

4 MCS and Block Error Rate (BLER) Model 7

5 HARQ with Chase Combining 8

6 OLLA: The Shared Outer Loop 9

7 Scheduling 9

II Markov Decision Process Formalization 11

8 Why OLLA-Relative Action Encoding 11

9 Reward 11

10 State Representation: The Two-Tier Design (D5) 11

11 User Classing for the Comparator Suite 12

III Algorithms and Their Complete Source 13

12 Tabular Q-Learning 13

13 Tabular Actor-Critic 14

14 Deep A2C 16
14.1 Behavior-Cloning Warm Start 16

IV The Comparator Suite 21

15 The Seven Schemes 21

16 Calibrating the Fixed-Offset Comparators 21

17 The Unified Evaluation Harness 22

V The Full v1–v7 Design Evolution 25

18 v1: Absolute Per-User Actions and the Tie-Breaking Bug 25

19 v2–v3: The No-Op Imitation Trap 25

20 v4: Baseline-Relative Action Rewrite 26

21 v5: An "Impossible" Result Reveals Two Deeper Bugs 26

22 v6: OLLA-Relative Encoding Closes the Information Gap 26
22.1 The v6 Lesson: The Constant-Offset Control 27

23 v7: The Five Design Levers, Restated as a Rationale 27

VI Experimental Protocol 30

24 Pre-Registration 30

25 The Comparison Harness 30

26 Point Presets and Campaign Design 34

VII Results 36

27 The Nine-Point Pre-Registered Campaign 36

28 Diagnosis: Five Root Causes for the Zero-Win Result **36**
28.1 Root Cause 1: Calibration/Training Distribution Asymmetry 36
28.2 Root Cause 2: Action-Set / Calibration-Grid Clipping 36
28.3 Root Cause 3: Undertraining on Cold Start 37
28.4 Root Cause 4: Entropy Restart After Behavior Cloning 37
28.5 Root Cause 5 (Ruled Out, Not Found): Missing State Information 37

29 Fix Iterations: `tier1`, `tier1b`, `tier1-nobc` **37**

30 Final Performance Summary: All Schemes, Final (`tier1b`) Model **38**

31 State Information Ablation (Tier A vs. Tier B) **38**

32 Adversarial Review and the OLLA Target-BLER Sweep **38**
32.1 The Objection 38
32.2 Why the Test Was Cheap 38
32.3 Results 40

33 Action Distribution: Direct Behavioral Evidence **41**

VIII Legacy Modules Appendix **42**

IX Limitations and Conclusion **43**

34 What This Investigation Does Not Establish **43**

35 Conclusion **43**

Part I Problem Statement and Physical-Layer System Model

1 The Link Adaptation Problem

A cellular base station serves N users on a shared downlink. Time is discretized into Transmission Time Intervals (TTIs) of duration $T_{\text{tti}} = 1\,\text{ms}$. In every TTI, for every user scheduled to transmit, the base station must choose a Modulation and Coding Scheme (MCS) $m \in \{0, \ldots, 14\}$ (15 MCS levels, LTE/NR-style), trading off raw bit rate against robustness to channel error. The achievable error-free rate for a given MCS depends on the instantaneous Signal-to-Interference-plus-Noise Ratio (SINR) at the receiver, which the base station does not observe directly: it only receives a delayed, quantized, noisy Channel Quality Indicator (CQI) report from the user equipment (UE). The central engineering trade-off, and the object of every method compared in this report, is: *choose the MCS that maximizes expected delivered throughput given only stale, imperfect channel information.*

1.1 Design Goals of the v7 Simulation

v7 is a deliberately re-engineered version of this simulation, built specifically to remove four simplifications present in the prior (v6) environment that were suspected – and later confirmed – to make constant, non-adaptive control policies artificially competitive with learned ones. Each design lever (D1–D5, defined below) targets one specific simplification. The full motivating history is given in Part V.

2 Propagation and Channel Model

2.1 Link Budget

For user i at 2D distance $d_{2D,i}(t)$ from the base station, the 3D distance incorporates the BS/UE height difference: $d_{3D,i}(t) = \sqrt{d_{2D,i}(t)^2 + (h_{BS} - h_{UE})^2}$. Path loss follows a 3GPP TR 38.901 Urban Macro Non-Line-of-Sight (UMa NLOS)-style model:

$$\text{PL}(d_{3D}) = 13.54 + 39.08 \log_{10}(d_{3D}) + 20 \log_{10}\left(\frac{f_c}{1\,\text{GHz}}\right) - 0.6\,(h_{UE} - 1.5) \quad [\text{dB}]$$

Shadow fading $SF(t)$ is modeled as a spatially-correlated AR(1) process driven by the user's own movement, with correlation distance $d_{corr} = 50\,\text{m}$: $SF(t) = r\,SF(t-1) + \sigma_{SF}\sqrt{1-r^2}\,\mathcal{N}(0,1)$, where $r = e^{-\Delta d(t)/d_{corr}}$ and $\Delta d(t)$ is the distance moved since the previous TTI.

Received power (dBm) is

$$P_{rx}(t) = P_{tx} + G_{dBi} - \text{PL}(d_{3D}(t)) - SF(t) + \text{Fad}_{dB}(t) - \text{O2I}_{dB}$$

and the noise floor is $N_0(\text{dBm}) = -174 + 10\log_{10}(BW) + NF_{dB}$. Instantaneous SINR is $P_{rx} - N_0 - IM_{dB} - I_{\text{extra}}(t)$, where IM_{dB} is a fixed implementation margin and $I_{\text{extra}}(t)$ is the burst-interference term (Section 2.3).

2.2 Small-Scale (Multipath) Fading: Jakes Model per User

v6 modeled multipath fading with a single, globally fixed AR(1) correlation coefficient, independent of how fast any given user was moving – physically incorrect, and (more importantly for the RL question) it silently discarded the coupling between "how fast is this user moving" and "how predictable is this user's channel," which is exactly the coupling a speed-aware policy would need to exploit. v7 (design lever **D4**) fixes this with

a standard Jakes/Clarke correlation model, deriving each user's instantaneous fading correlation directly from their own speed via the Doppler relation:

$$\rho_i(t) = J_0\left(2\pi \frac{v_i(t)\, f_c}{c}\, T_{\text{tti}}\right)$$

where J_0 is the zeroth-order Bessel function of the first kind (`besselj(0,...)` in MATLAB) – the standard closed-form autocorrelation of Rayleigh fading under 2D isotropic scattering. The complex channel gain follows an AR(1) recursion with this *per-user, per-TTI* coefficient: $h_i(t) = \rho_i(t)\, h_i(t-1) + \sqrt{1-\rho_i(t)^2}\, \mathcal{CN}(0,1)$, and $\text{Fad}_{dB}(t) = 10\log_{10}(|h_i(t)|^2)$.

2.3 Interference Bursts (Non-Stationarity)

Design lever **D3**. A cell-wide, two-state Markov chain toggles extra interference on and off, with mean dwell times `burst_dwell_calm` (default 250 TTI) and `burst_dwell_burst` (default 120 TTI), i.e. transition probabilities $p_{\text{enter}} = 1/250$, $p_{\text{exit}} = 1/120$ per TTI. While active, $I_{\text{extra}}(t) =$ burst_IM_dB (default 9 dB) is added to every user's interference term. Independently, each user has a 50% chance of a single mid-trace speed change at a uniformly random point in $[0.3, 0.7] \times$ simTime, redrawn from the same heterogeneity distribution.

2.4 Full Channel Generator Source

```
function [SINR_dB, dist, meta] = generateChannel_v7(simTime, env, metaFix)
% generateChannel_v7 v7 channel generator: heterogeneous users, Jakes
% fading, interference bursts, mid-trace speed changes.
    if nargin < 3, metaFix = []; end
    N = env.N;

    if ~isempty(metaFix)
        delays  = metaFix.delays;
        baseSpd = metaFix.baseSpeed;
        spd2    = metaFix.speed2;
        switchT = min(metaFix.switchT, simTime);
    else
        h = max(0, min(1, env.hetero));
        if h == 0
            delays = env.delay_hom * ones(1, N);
            baseSpd = env.speed * ones(1, N);
        else
            ds = env.delay_set;
            mid = env.delay_hom;
            lo  = round(mid - h*(mid - min(ds))); hi = round(mid + h*(max(ds) - mid));
            delays = lo + randi(hi - lo + 1, 1, N) - 1;
            baseSpd = env.speed_min + h*(env.speed_max - env.speed_min)*rand(1, N);
        end
        spd2 = baseSpd;
        switchT = (simTime+1) * ones(1, N);
        if isfield(env,'speed_switch') && env.speed_switch
            for i = 1:N
                if rand < 0.5
                    switchT(i) = round(simTime*(0.3 + 0.4*rand));
                    if h == 0, spd2(i) = baseSpd(i);
                    else, spd2(i) = env.speed_min + h*(env.speed_max - env.speed_min)*rand;
                    end
                end
            end
        end
    end

    speeds = zeros(simTime, N);
    for i = 1:N
        speeds(:, i) = baseSpd(i);
        if switchT(i) <= simTime
            speeds(switchT(i):end, i) = spd2(i);
        end
```

```
    end

    % Jakes AR(1) coefficient per user per TTI
    if isfield(env,'jakes') && env.jakes
        fd  = speeds * env.fc / 3e8;
        rho = besselj(0, 2*pi*fd*env.T_tti);
        rho = max(0, min(0.9999, rho));
    else
        rho = env.rho_fading * ones(simTime, N);
    end

    % Interference burst chain (cell-wide 2-state Markov)
    burst = false(simTime, 1);
    if isfield(env,'burst_on') && env.burst_on
        pEnter = 1/max(env.burst_dwell_calm, 1);
        pExit  = 1/max(env.burst_dwell_burst, 1);
        b = false;
        for t = 1:simTime
            if b, if rand < pExit, b = false; end
            else, if rand < pEnter, b = true; end
            end
            burst(t) = b;
        end
    end
    Iextra = double(burst) * env.burst_IM_dB;

    % Mobility (RWP) + link budget
    uePos = zeros(simTime, 2, N);
    for i = 1:N
        pos  = env.areaSize * rand(1, 2);
        dest = env.areaSize * rand(1, 2);
        for t = 1:simTime
            v = speeds(t, i);
            dir = dest - pos; L = norm(dir);
            if L < v, pos = dest; dest = env.areaSize * rand(1, 2);
            else,     pos = pos + v * dir / L;
            end
            uePos(t, :, i) = pos;
        end
    end

    SINR_dB = zeros(simTime, N);
    dist    = zeros(simTime, N);
    noise_dBm = -174 + 10*log10(env.BW) + env.NF_dB;

    for i = 1:N
        d2D = sqrt((uePos(:,1,i) - env.bsPos(1)).^2 + (uePos(:,2,i) - env.bsPos(2)).^2);
        d2D = max(d2D, 10);
        dist(:, i) = d2D;
        d3D = sqrt(d2D.^2 + (env.hBS - env.hUE)^2);
        PL = 13.54 + 39.08*log10(d3D) + 20*log10(env.fc/1e9) - 0.6*(env.hUE - 1.5);

        SF = zeros(simTime, 1); SF(1) = env.sigma_SF * randn;
        step_d = [0; sqrt(sum(diff(squeeze(uePos(:,:,i))).^2, 2))];
        for t = 2:simTime
            r = exp(-step_d(t) / env.d_corr);
            SF(t) = r*SF(t-1) + env.sigma_SF*sqrt(1 - r^2)*randn;
        end

        h = zeros(simTime, 1); h(1) = (randn + 1i*randn)/sqrt(2);
        for t = 2:simTime
            rf = rho(t, i);
            h(t) = rf*h(t-1) + sqrt(1 - rf^2)*(randn + 1i*randn)/sqrt(2);
        end
        fad_dB = 10*log10(abs(h).^2 + 1e-12);

        rx_dBm = env.txPower_dBm + env.G_dBi - PL - SF + fad_dB - env.O2I_dB;
        SINR_dB(:, i) = rx_dBm - noise_dBm - env.IM_dB - Iextra;
    end
```

```
    meta = struct('delays', delays, 'speeds', speeds, 'rho', rho, ...
                  'burst', burst, 'baseSpeed', baseSpd, 'speed2', spd2, ...
                  'switchT', switchT);
end
```

Listing 1: `generateChannel_v7.m` — complete channel generator: heterogeneous per-user delay/speed draws (D2), mid-trace speed switch (D3), Jakes fading (D4), interference burst chain (D3), random-waypoint mobility, and the full link budget.

3 CQI Feedback Model

The UE measures true SINR and reports a delayed, noisy version back to the base station. Each user i has its own feedback delay d_i (heterogeneity lever D2, drawn from `env.delay_set` = 2–8 TTI) and the reported SINR is corrupted by additive measurement noise:

$$\widehat{\mathrm{SINR}}_i(t) = \mathrm{SINR}_i(t - d_i) + \mathcal{N}(0, \sigma^2_{\mathrm{meas}}), \qquad \sigma_{\mathrm{meas}} = 1.0\,\mathrm{dB}$$

The observed SINR is then quantized to a CQI index 1–15 via a threshold table anchored at the BLER = 0.1 point of each MCS's waterfall curve (Section 4).

```
function [obs_SINR, obs_CQI] = cqiFeedback_v7(SINR_dB, env, delays)
% cqiFeedback_v7 Per-user delayed, noisy CQI feedback chain.
    [T, N] = size(SINR_dB);
    if nargin < 3 || isempty(delays), delays = env.cqi_delay * ones(1, N); end

    obs_SINR = zeros(T, N);
    for i = 1:N
        idx = max((1:T).' - delays(i), 1);
        obs_SINR(:, i) = SINR_dB(idx, i);
    end
    obs_SINR = obs_SINR + env.sigma_meas * randn(T, N);

    if nargout > 1
        obs_CQI = zeros(T, N);
        for i = 1:N
            for t = 1:T
                q = find(obs_SINR(t, i) >= env.CQI_thresholds, 1, 'last');
                if isempty(q), q = 1; end
                obs_CQI(t, i) = max(1, min(15, q));
            end
        end
    end
end
```

Listing 2: `cqiFeedback_v7.m` — per-user delayed, noisy CQI chain.

4 MCS and Block Error Rate (BLER) Model

Design lever **D4**. For a given MCS m and instantaneous SINR, BLER follows a logistic "waterfall" curve:

$$\mathrm{BLER}(\mathrm{SINR}, m) = \frac{1}{1 + \exp\big(k_m(\mathrm{SINR} - c_m)\big)}, \qquad c_m = \theta_m - \frac{\ln 9}{k_m}$$

where θ_m is the SINR at which BLER = 0.1 (matching the CQI threshold table, so the CQI mapping and the BLER model stay mutually consistent) and k_m is the curve's steepness, rising from 3.0 (MCS 0) to 5.8 (MCS 14). This is the key physical change from v5/v6, whose curves used a single shallow slope family ($k = 1.2$–2.7) for every MCS. A shallow curve makes BLER a soft, forgiving function of MCS error – the throughput-optimal operating BLER under v6's curves sat near 0.45 *regardless of context*, so any policy that simply ran "hot" (aggressive MCS choice) did about as well as any other. Steepening the curve, combined with HARQ (Section 5)

making failures costly, restores a genuine trade-off: one MCS step too high collapses the success probability; one step too low wastes capacity. Getting this choice right, conditioned on context, is precisely the job description of a learned policy – which is why this change matters for the RL-vs-baseline question, not just for physical realism.

```
function BLER = estimateBLER_v7(sinr_dB, mcs)
% estimateBLER_v7 Per-MCS BLER waterfall: BLER = 1/(1+exp(k_m*(sinr-c_m))).
    persistent th k c
    if isempty(th)
        mcs_params = [ ...
            -6.7, 3.0; -5.0, 3.2; -3.2, 3.4; -1.5, 3.6; 0.8, 3.8; ...
             2.0, 4.0; 4.5, 4.2; 6.8, 4.4; 9.2, 4.6; 11.5, 4.8; ...
            14.0, 5.0; 16.3, 5.2; 18.7, 5.4; 20.5, 5.6; 21.8, 5.8];
        th = mcs_params(:, 1); k = mcs_params(:, 2);
        c  = th - log(9) ./ k;
    end
    m = max(0, min(14, round(mcs))) + 1;
    BLER = 1 ./ (1 + exp(k(m) .* (sinr_dB - c(m))));
    BLER = max(0, min(1, BLER));
end
```

Listing 3: `estimateBLER_v7.m` — steep per-MCS waterfall BLER model (D4).

5 HARQ with Chase Combining

Design lever **D1**. In v6, a failed transmission was scored as a probabilistic discount on throughput with no further consequence – there was no mechanism by which failure cost anything beyond its own probability. This creates a degenerate optimum: since failure has no extra penalty, the throughput-maximizing strategy is simply "run as hot as possible, let the average work out," which never rewards situational restraint.

v7 replaces this with Hybrid Automatic Repeat reQuest (HARQ) with chase combining. A failed transmission is not discarded: the receiver keeps the corrupted soft bits and combines them with a retransmission of the *same* transport block, at the *same* MCS, with an effective SINR boost of `harq_comb_dB` = 2.5 dB per prior attempt (up to `harq_max_retx`= 3 retries, after which the block is dropped). Crucially, a retransmission consumes a real time slot that could otherwise have carried new data for someone else – so an overly aggressive first attempt now has a genuine, measurable cost (wasted retransmission capacity), creating the "transmit fast and risk a costly retry" vs. "transmit conservatively and use every slot for new data" trade-off that a context-aware policy could in principle navigate better than a fixed rule.

OLLA's outer-loop offset (Section 6) is updated only on the *first-transmission* ACK/NACK – standard outer-loop behavior, and identical logic is shared by every learner and comparator via the single function below.

```
function H = harqInit_v7(N)
% harqInit_v7 Fresh per-user HARQ state.
    H = struct('pend', false(1, N), 'nret', zeros(1, N), ...
               'tbs', zeros(1, N), 'mcs', zeros(1, N));
end
```

Listing 4: `harqInit_v7.m` — per-user HARQ state initializer.

```
function [tp, off, H, bler1, nackEv] = txStep_v7(sent, Strue_row, rb, off, H, env)
% txStep_v7 One TTI of transmission physics: HARQ, Bernoulli delivery, OLLA update.
    N = numel(sent);
    tp = zeros(1, N); bler1 = nan(1, N); nackEv = false(1, N);
    du = env.olla_step_down * env.olla_target/(1 - env.olla_target);
    dd = env.olla_step_down;

    for i = 1:N
        if rb(i) <= 0, continue; end

        if env.harq_on && H.pend(i)
            % Retransmission (chase combining), no OLLA update
            effS = Strue_row(i) + env.harq_comb_dB * H.nret(i);
```

```
            bl   = estimateBLER_v7(effS, H.mcs(i));
            if rand > bl
                tp(i) = H.tbs(i) * 1e-3;
                H.pend(i) = false; H.nret(i) = 0;
            else
                H.nret(i) = H.nret(i) + 1;
                if H.nret(i) > env.harq_max_retx
                    H.pend(i) = false; H.nret(i) = 0;
                end
            end
        else
            % New transmission at the chosen CQI
            mcs = sent(i) - 1;
            bl  = estimateBLER_v7(Strue_row(i), mcs);
            bler1(i) = bl;
            TBS = tbsFromMCSandRB(mcs, rb(i));
            ack = rand > bl;
            if ack
                tp(i) = TBS * 1e-3;
                off(i) = off(i) + du;
            else
                nackEv(i) = true;
                off(i) = off(i) - dd;
                if env.harq_on
                    H.pend(i) = true; H.nret(i) = 1;
                    H.tbs(i) = TBS; H.mcs(i) = mcs;
                end
            end
        end
    end
end
```

Listing 5: `txStep_v7.m` — the single shared transmission-physics function: HARQ chase combining, Bernoulli delivery, and the OLLA update. Used identically by every trainer and by the evaluation harness, so training and evaluation dynamics are guaranteed identical.

6 OLLA: The Shared Outer Loop

Outer Loop Link Adaptation (OLLA) is the classical, industry-standard control algorithm this project compares against. It is a single-parameter integral controller: it maintains a running offset Δ_{OLLA} added to the reported SINR before mapping to CQI, and updates it after every first-transmission outcome (implemented inside `txStep_v7.m`, lines computing `du`/`dd` and the `off(i)= off(i)+ du` / `off(i)= off(i)- dd` branches shown in Listing 5):

$$\Delta_{\text{OLLA}} \leftarrow \begin{cases} \Delta_{\text{OLLA}} + \delta_{\text{up}} & \text{ACK} \\ \Delta_{\text{OLLA}} - \delta_{\text{down}} & \text{NACK} \end{cases}, \qquad \delta_{\text{up}} = \delta_{\text{down}} \cdot \frac{P_{\text{target}}}{1 - P_{\text{target}}}$$

with $P_{\text{target}} = 0.1$ (`env.olla_target`) and $\delta_{\text{down}} = 0.5$ (`env.olla_step_down`) by default. This balance condition is exactly what makes the up/down rule converge in steady state to the target error rate. Critically, **every** scheme in this report except the raw `Baseline` carries this identical OLLA loop internally (see `useOLLA` in `evalPolicy_v7.m`, Listing 18) – OLLA is not just a comparator, it is the shared reference frame every RL action is defined relative to (Part II).

7 Scheduling

Every TTI, a scheduler decides which user(s) get resource blocks (RBs). Three scheduler policies are implemented and all are used somewhere in the campaign (PF is the pre-registered primary; RR and EQ are secondary/training-mixture schedulers):

- **Round Robin (RR)**: cycles through users one per TTI, full bandwidth each.

- **Proportional Fair (PF)**: schedules $\arg\max_i \text{potential}_i(t) / \max(R_{\text{ema},i}, 0.1)$, i.e. the user whose *currently achievable* rate is largest relative to their own recent average throughput R_{ema} (exponential moving average, $\alpha = 0.05$). This is the industry-standard fairness-throughput balance and the pre-registered primary scheduler (Section 24).
- **Equal share (EQ)**: splits RBs evenly across all N users every TTI.

An important, audited property (inherited from the v5/v6 line and re-verified for v7): the PF metric uses only the *reported*, potential rate (`env.pf_metric = 'reported'`) – i.e. exactly what a real scheduler would have access to at decision time, never the true, unobserved channel. This was originally an audit fix (see §21) that closed a scheduler information leak.

```
function rb_alloc = schedule_users_v5(mode, t, N, R_emas, potential_rates, total_RBs)
% schedule_users_v5 RB allocator: 'all' | 'rr' | 'pf' | 'eq'.
    rb_alloc = zeros(1, N);
    switch lower(mode)
        case 'all'
            rb_alloc(1) = total_RBs;
        case {'rr', 'roundrobin'}
            rb_alloc(mod(t-1, N) + 1) = total_RBs;
        case {'pf', 'proportionalfair', 'dynamic'}
            [~, served] = max(potential_rates ./ max(R_emas, 0.1));
            rb_alloc(served) = total_RBs;
        case {'eq', 'equal', 'share'}
            base = floor(total_RBs / N); rb_alloc(:) = base;
            rem = total_RBs - sum(rb_alloc);
            if rem > 0, rb_alloc(1:rem) = rb_alloc(1:rem) + 1; end
        otherwise
            error('schedule_users_v5: unknown mode "%s"', mode);
    end
end
```

Listing 6: `schedule_users_v5.m` — resource-block allocator (RR / PF / EQ), reused unchanged from v5.

```
function [code, onehot] = schedContext_v5(mode, env)
% schedContext_v5 Map a scheduler mode to an observable one-hot context.
    nS = numel(env.sched_list);
    code = find(strcmpi(env.sched_list, mode), 1);
    if isempty(code), code = 1; end
    onehot = zeros(nS, 1);
    onehot(code) = 1;
end
```

Listing 7: `schedContext_v5.m` — maps the active scheduler to an observable one-hot context, appended to every learner's state so a single shared policy can specialize per scheduling regime.

```
function mode = sampleSched_v5(env)
% sampleSched_v5 Draw a training scheduler using env.train_sched_weights.
    w = env.train_sched_weights;
    if isempty(w) || numel(w) ~= numel(env.train_schedulers)
        w = ones(1, numel(env.train_schedulers));
    end
    w = w / sum(w);
    idx = find(rand <= cumsum(w), 1, 'first');
    if isempty(idx), idx = numel(env.train_schedulers); end
    mode = env.train_schedulers{idx};
end
```

Listing 8: `sampleSched_v5.m` — draws a training-time scheduler according to `env.train_sched_weights`. This weighting is one of the three Tier-1 fixes (§29).

Part II Markov Decision Process Formalization

8 Why OLLA-Relative Action Encoding

If an RL agent chose an MCS directly, independent of what OLLA would have chosen, the comparison would be structurally unfair: OLLA has access to fast, per-transmission ACK/NACK feedback that a naively-designed agent state might not expose, so any observed "RL loses to OLLA" result would measure an information deficit, not an algorithmic one. The fix, first introduced in v6 (§22) and retained throughout v7, is to give *every* learner and comparator the identical OLLA loop internally, and define the RL action as an additive correction on top of it:

$$\text{chosen CQI} = \text{clip}\big(\text{OLLA-corrected reference CQI} + a_t,\ \text{min_CQI},\ \text{max_CQI}\big), \qquad a_t \in \{-K, \ldots, -1, 0, 1, \ldots, K\}$$

Action $a_t = 0$ makes the policy behave *exactly* like plain OLLA. This guarantees, by construction, that RL can never do worse than OLLA in principle (it can always learn to emit 0), and it re-frames the experimental question precisely: *does the agent learn anything beyond what OLLA already does*, rather than conflating that question with "does the agent have OLLA's basic feedback." The action set used throughout the v7 headline campaign is `env.actions = -3:3` (widened to `-5:3` in the Tier-1 fixes, §29).

9 Reward

The reward is the realized, delivered cell throughput at each TTI, pooled cooperatively across all N users and scaled:

$$r_t = \frac{1}{\text{reward_scale}} \sum_{i=1}^{N} \text{tp}_i(t)$$

where $\text{tp}_i(t)$ is user i's realized (Bernoulli, HARQ-aware) delivered goodput from `txStep_v7.m` (Listing 5). Using a single, shared, cell-level reward (rather than per-user rewards) is a design choice inherited from an earlier bug fix (§21): per-user rewards created a multi-agent incentive-misalignment problem where a user could benefit from mis-reporting its own channel quality to win more scheduling opportunities.

10 State Representation: The Two-Tier Design (D5)

Design lever **D5** is not a physical change to the environment – it is an experimental control targeting a different possible confound: is any RL advantage (or disadvantage) actually explained by *what information the agent was given*, rather than by whether adaptivity has any value at all? v7 defines two observation tiers, selected via `env.state_tier`:

- **Tier A** (information-equal): exactly what OLLA's own loop has access to – reported CQI history, its recent change, current OLLA offset, plus scheduler context. $5 + n_{\text{sched}}$ features.
- **Tier B** (full state, the v7 headline): Tier A plus five additional features no scalar control loop could represent even in principle – a 4-TTI observed-SINR slope, an EMA of first-transmission NACK events (the same events OLLA itself reacts to, just used nonlinearly), HARQ pending/retry-count state, the user's instantaneous speed, and a delayed, per-user-observed interference-burst indicator. $11 + n_{\text{sched}}$ features.

Running the identical experiment under both tiers and comparing (§31) isolates the "does richer information help" question from every other question in this report.

```
function S = stateFeat_v7(tier, ref, dRef, obs, dS, off, slope4, nk, H, spd, burstObs, ctx)
% stateFeat_v7 Deep-agent feature vector, Tier A (info-equal) or Tier B (full state).
    base = [ref/15; dRef/5; obs/30; dS/5; off/5];
    if upper(tier) == 'A'
        S = single([base; ctx]);
    else
        extra = [slope4/5; nk; double(H.pend); H.nret/3; spd/30; burstObs];
        S = single([base; extra; ctx]);
    end
end
```

Listing 9: `stateFeat_v7.m` — constructs the Tier-A or Tier-B feature vector used by the deep agent.

11 User Classing for the Comparator Suite

The oracle comparator (FixedPU, §15) is calibrated per *class* of user rather than globally, using a coarse 2×3 grid on (feedback delay, mean speed). The exact same classing function is used both to calibrate FixedPU and to score it, guaranteeing the comparator is defined identically everywhere it is used.

```
function [dc, vc] = userClass_v7(delay, meanSpeed)
% userClass_v7 Map (delay, mean speed) to the FixedPU class grid.
%   dc: 1 = delay <= 4, 2 = delay >= 5
%   vc: 1 = < 10 m/s, 2 = 10-20 m/s, 3 = > 20 m/s
    dc = 1 + (delay >= 5);
    vc = 1 + (meanSpeed >= 10) + (meanSpeed > 20);
end
```

Listing 10: `userClass_v7.m` — maps a user's (delay, mean speed) onto the FixedPU class grid.

Part III Algorithms and Their Complete Source

All three learning algorithms below share: (1) the identical v7 environment physics (`txStep_v7.m`, `generateChannel_v7.m`, `cqiFeedback_v7.m`); (2) the identical OLLA-relative action encoding (§8); (3) the identical scheduler-context augmentation (§7); and (4) the identical cooperative cell-level reward. They differ only in function-approximation capacity and update rule – exactly the axis this report's central question is about.

12 Tabular Q-Learning

Shared-parameter (all users share one table), Tier-A-state-only tabular Q-learning with ε-greedy exploration decaying geometrically from $\varepsilon = 0.30$ to 0.05 over training, learning rate $\alpha = 0.1$, discount $\gamma = 0.95$. The Q-table is indexed by (reference CQI, discretized CQI delta, SINR bin, trend bin, discretized OLLA-offset bin, scheduler context, action) – Tier B's continuous/HARQ features are deliberately not representable in this table, which is itself part of the capacity story (a tabular method cannot even in principle use Tier-B information). Ties in the greedy action are broken toward the no-op action (offset 0) by `argmaxNoOp_v5.m` – a fix inherited from the v1 tie-breaking bug (§18).

```
function [Q, learning_curve] = QL_Shared_v7(simTime, num_episodes, seed, env)
% QL_Shared_v7 Shared-parameter tabular Q-learning (Tier-A state).
    if ~isempty(seed), rng(seed); end
    N = env.N; A = env.actions; nA = numel(A); no_op = find(A == 0);
    SB = env.SINR_bins; nT = env.n_trend; te = env.trend_edges;
    maxC = env.max_CQI; minC = env.min_CQI; maxD = env.max_delta; minD = env.min_delta;
    OE = env.off_edges; nO = numel(OE) + 1;

    alpha = 0.1; gamma = env.gamma;
    epsilon = 0.30; eps_decay = (0.05/0.30)^(1/max(num_episodes,1));
    nSched = numel(env.sched_list);

    Q = zeros(maxC, maxD-minD+1, numel(SB), nT, nO, nSched, nA);
    learning_curve = zeros(num_episodes, 1);

    for ep = 1:num_episodes
        sched = sampleSched_v5(env);
        sc = schedContext_v5(sched, env);
        [Strue, ~, meta] = generateChannel_v7(simTime, env);
        obsS = cqiFeedback_v7(Strue, env, meta.delays);
        off = zeros(1, N); H = harqInit_v7(N);
        prevRef = baseCQI_v5(obsS(1,:) + off, env);
        R_emas = zeros(1, N); ema = 0.05; ep_tp = 0;

        for t = 1:simTime
            ref  = baseCQI_v5(obsS(t,:) + off, env);
            dRef = max(minD, min(maxD, ref - prevRef));
            aIdx = zeros(1,N); sent = zeros(1,N); pot = zeros(1,N);
            si = zeros(1,N); tb = zeros(1,N); ob = zeros(1,N);
            for i = 1:N
                [~, si(i)] = min(abs(SB - obsS(t,i)));
                tb(i) = trendBin_v5(obsS(t,i) - obsS(max(t-1,1),i), te);
                ob(i) = 1 + sum(off(i) > OE);
                sd = dRef(i) + maxD + 1;
                if rand < epsilon, aIdx(i) = randi(nA);
                else, aIdx(i) = argmaxNoOp_v5(squeeze(Q(ref(i),sd,si(i),tb(i),ob(i),sc,:)), no_op); end
                sent(i) = max(minC, min(maxC, ref(i) + A(aIdx(i))));
                if env.harq_on && H.pend(i)
                    pot(i) = tbsFromMCSandRB(H.mcs(i), env.total_RB) * 1e-3;
```

```
            else
                pot(i) = tbsFromMCSandRB(sent(i)-1, env.total_RB) * 1e-3;
            end
        end

        rb = schedule_users_v5(sched, t, N, R_emas, pot, env.total_RB);
        [tp_now, off, H] = txStep_v7(sent, Strue(t,:), rb, off, H, env);
        R_emas = (1-ema)*R_emas + ema*tp_now; ep_tp = ep_tp + sum(tp_now);
        r_cell = sum(tp_now) / env.reward_scale;

        tn = min(t+1, simTime);
        refN  = baseCQI_v5(obsS(tn,:) + off, env);
        dRefN = max(minD, min(maxD, refN - ref));
        for i = 1:N
            r = r_cell;
            [~, nsi] = min(abs(SB - obsS(tn,i)));
            ntb = trendBin_v5(obsS(tn,i) - obsS(t,i), te);
            nob = 1 + sum(off(i) > OE);
            sd = dRef(i) + maxD + 1;
            maxQ = max(Q(refN(i), dRefN(i)+maxD+1, nsi, ntb, nob, sc, :));
            Q(ref(i),sd,si(i),tb(i),ob(i),sc,aIdx(i)) = Q(ref(i),sd,si(i),tb(i),ob(i),sc,aIdx(i)) + ...
                alpha*(r + gamma*maxQ - Q(ref(i),sd,si(i),tb(i),ob(i),sc,aIdx(i)));
        end
        prevRef = ref;
    end
    learning_curve(ep) = ep_tp / simTime;
    epsilon = epsilon * eps_decay;
  end
end
```

Listing 11: `QL_Shared_v7.m` — complete tabular Q-learning trainer.

```
function a = argmaxNoOp_v5(q, no_op_idx)
% argmaxNoOp_v5 Greedy action, ties broken toward the no-op action.
    q = q(:); m = max(q); ties = find(q >= m - 1e-12);
    if any(ties == no_op_idx), a = no_op_idx; else, a = ties(1); end
end
```

Listing 12: `argmaxNoOp_v5.m` — greedy action selection with ties broken toward the no-op action.

```
function cqi = baseCQI_v5(obs_SINR_dB, env)
% baseCQI_v5 Fixed SINR-to-CQI mapping; the reference every action is relative to.
    cqi = zeros(size(obs_SINR_dB));
    for j = 1:numel(obs_SINR_dB)
        q = find(obs_SINR_dB(j) >= env.CQI_thresholds, 1, 'last');
        if isempty(q), q = env.min_CQI; end
        cqi(j) = max(env.min_CQI, min(env.max_CQI, q));
    end
end
```

Listing 13: `baseCQI_v5.m` — fixed SINR-to-CQI mapping; the reference point every action is relative to.

```
function b = trendBin_v5(dS, edges)
% trendBin_v5 3-bin SINR-trend discretization: 1=falling, 2=flat, 3=rising.
    if dS < edges(1), b = 1; elseif dS > edges(2), b = 3; else, b = 2; end
end
```

Listing 14: `trendBin_v5.m` — 3-bin discretization of the observed-SINR trend (falling/flat/rising).

13 Tabular Actor-Critic

Same Tier-A state space and table structure as Q-learning, but with an explicit softmax policy table θ and a separate state-value table V, updated by the standard one-step TD actor-critic rule: $\delta_{TD} = r + \gamma V(s') - V(s)$;

$V(s) \leftarrow V(s) + \alpha_v \delta_{TD}$; $\theta(s,\cdot) \leftarrow \theta(s,\cdot) + \alpha_\theta \delta_{TD}\,(\mathbf{1}_a - \pi(\cdot|s))$. Learning rates $\alpha_\theta = 0.05$, $\alpha_v = 0.10$.

```
function [theta, V, learning_curve] = AC_Shared_v7(simTime, num_episodes, seed, env)
% AC_Shared_v7 Shared-parameter tabular actor-critic (Tier-A state).
    if ~isempty(seed), rng(seed); end
    N = env.N; A = env.actions; nA = numel(A);
    SB = env.SINR_bins; nT = env.n_trend; te = env.trend_edges;
    maxC = env.max_CQI; minC = env.min_CQI; maxD = env.max_delta; minD = env.min_delta;
    OE = env.off_edges; nO = numel(OE) + 1;

    alpha_theta = 0.05; alpha_v = 0.10; gamma = env.gamma;
    nSched = numel(env.sched_list);

    theta = zeros(maxC, maxD-minD+1, numel(SB), nT, nO, nSched, nA);
    V     = zeros(maxC, maxD-minD+1, numel(SB), nT, nO, nSched);
    learning_curve = zeros(num_episodes, 1);

    for ep = 1:num_episodes
        sched = sampleSched_v5(env);
        sc = schedContext_v5(sched, env);
        [Strue, ~, meta] = generateChannel_v7(simTime, env);
        obsS = cqiFeedback_v7(Strue, env, meta.delays);
        off = zeros(1, N); H = harqInit_v7(N);
        prevRef = baseCQI_v5(obsS(1,:) + off, env);
        R_emas = zeros(1, N); ema = 0.05; ep_tp = 0;

        for t = 1:simTime
            ref  = baseCQI_v5(obsS(t,:) + off, env);
            dRef = max(minD, min(maxD, ref - prevRef));
            aIdx = zeros(1,N); sent = zeros(1,N); pot = zeros(1,N);
            sidx = zeros(N,5); pis = cell(1,N);
            for i = 1:N
                [~, sb] = min(abs(SB - obsS(t,i)));
                tbi = trendBin_v5(obsS(t,i) - obsS(max(t-1,1),i), te);
                obn = 1 + sum(off(i) > OE);
                sd  = dRef(i) + maxD + 1;
                sidx(i,:) = [ref(i), sd, sb, tbi, obn];
                prefs = squeeze(theta(ref(i), sd, sb, tbi, obn, sc, :));
                prefs = prefs - max(prefs);
                p = exp(prefs)/sum(exp(prefs)); pis{i} = p;
                a = find(rand <= cumsum(p), 1, 'first');
                if isempty(a), a = nA; end
                aIdx(i) = a;
                sent(i) = max(minC, min(maxC, ref(i) + A(a)));
                if env.harq_on && H.pend(i)
                    pot(i) = tbsFromMCSandRB(H.mcs(i), env.total_RB) * 1e-3;
                else
                    pot(i) = tbsFromMCSandRB(sent(i)-1, env.total_RB) * 1e-3;
                end
            end

            rb = schedule_users_v5(sched, t, N, R_emas, pot, env.total_RB);
            [tp_now, off, H] = txStep_v7(sent, Strue(t,:), rb, off, H, env);
            R_emas = (1-ema)*R_emas + ema*tp_now; ep_tp = ep_tp + sum(tp_now);
            r_cell = sum(tp_now) / env.reward_scale;

            tn = min(t+1, simTime);
            refN  = baseCQI_v5(obsS(tn,:) + off, env);
            dRefN = max(minD, min(maxD, refN - ref));
            for i = 1:N
                r = r_cell;
                [~, nsb] = min(abs(SB - obsS(tn,i)));
                ntb = trendBin_v5(obsS(tn,i) - obsS(t,i), te);
                nob = 1 + sum(off(i) > OE);
                s1=sidx(i,1); s2=sidx(i,2); s3=sidx(i,3); s4=sidx(i,4); s5=sidx(i,5);
                V_s  = V(s1,s2,s3,s4,s5,sc);
                V_sp = V(refN(i), dRefN(i)+maxD+1, nsb, ntb, nob, sc);
                td   = r + gamma*V_sp - V_s;
                V(s1,s2,s3,s4,s5,sc) = V_s + alpha_v*td;
```

```
            g = -pis{i}; g(aIdx(i)) = g(aIdx(i)) + 1;
            theta(s1,s2,s3,s4,s5,sc,:) = squeeze(theta(s1,s2,s3,s4,s5,sc,:)) + alpha_theta*td*g;
        end
        prevRef = ref;
    end
    learning_curve(ep) = ep_tp / simTime;
  end
end
```

Listing 15: `AC_Shared_v7.m` — complete tabular actor-critic trainer.

14 Deep A2C

The primary learner. A shared-parameter, feed-forward n-step Advantage Actor-Critic with two independent networks (actor: state $\rightarrow$ action logits; critic: state $\rightarrow$ value), each a 2-hidden-layer MLP with 96 ReLU units per layer, trained with Adam ($\mathrm{lr}_{\mathrm{actor}} = 5 \times 10^{-4}$, $\mathrm{lr}_{\mathrm{critic}} = 10^{-3}$), n-step returns ($n = 10$ TTIs $\times$ N users per rollout buffer), global gradient-norm clipping (clip = 5.0), per-batch advantage standardization, and an entropy-bonus schedule annealing from 0.05 to 0.001 across training. State tier is selectable (Tier A or Tier B, §10); the headline configuration uses Tier B.

14.1 Behavior-Cloning Warm Start

If `env.bc_warm_eps` > 0, the first `bc_warm_eps` episodes do not run standard RL rollouts. Instead, each scheduled action is forced to FixedPU's calibrated per-class constant offset (clipped to the action set), and the actor is trained with cross-entropy loss against those target actions (plus a small entropy bonus, `entW_bc=0.03`, to keep the resulting policy soft rather than fully saturated), while the critic regresses the observed n-step returns of the cloned rollout (a value warm start). Standard RL fine-tuning then begins from a policy that already reproduces the oracle comparator, so any positive margin measured *after* fine-tuning is attributable to state-conditional value discovered beyond imitation, not to optimization noise. The exact mechanism by which an earlier, naively-scheduled version of this warm start *failed* (the entropy-restart bug) is detailed in §28.4.

```
function [actorNet, criticNet, learning_curve, entropy_curve] = ...
          DeepA2C_Shared_v7(simTime, num_episodes, seed, env)
% DeepA2C_Shared_v7 Shared-network n-step A2C (v7 environment).
% If env.bc_warm_eps > 0, the first bc_warm_eps episodes behavior-clone
% the FixedPU comparator before RL fine-tuning begins.
    if ~isempty(seed), rng(seed); end
    N = env.N; A = env.actions; nA = numel(A);
    maxC = env.max_CQI; minC = env.min_CQI; maxD = env.max_delta; minD = env.min_delta;

    gamma = env.gamma; n_step = 10;
    lr_actor = 5e-4; lr_critic = 1e-3;
    ent_start = 0.05; ent_end = 0.001; clip_norm = 5.0;

    nSched = numel(env.sched_list);
    if upper(env.state_tier) == 'A', nBase = 5; else, nBase = 11; end
    nFeat = nBase + nSched;

    actorLayers = [
        featureInputLayer(nFeat, 'Name', 'in')
        fullyConnectedLayer(96, 'Name', 'fc1'); reluLayer('Name','r1')
        fullyConnectedLayer(96, 'Name', 'fc2'); reluLayer('Name','r2')
        fullyConnectedLayer(nA, 'Name', 'logits')];
    criticLayers = [
        featureInputLayer(nFeat, 'Name', 'in')
        fullyConnectedLayer(96, 'Name', 'fc1'); reluLayer('Name','r1')
        fullyConnectedLayer(96, 'Name', 'fc2'); reluLayer('Name','r2')
        fullyConnectedLayer(1, 'Name', 'value')];
    actorNet = dlnetwork(actorLayers);
    criticNet = dlnetwork(criticLayers);
    toDl = @(x) dlarray(single(x), 'CB');

```

```
    aAvg=[]; aAvgSq=[]; aIt=0; cAvg=[]; cAvgSq=[]; cIt=0;
    learning_curve = zeros(num_episodes,1); entropy_curve = zeros(num_episodes,1);

    cap = n_step * N;
    bufS = zeros(nFeat, cap, 'single'); bufA = zeros(1, cap); bufR = zeros(1, cap, 'single');
    nb = 0;

    bcEps = 0;
    if isfield(env, 'bc_warm_eps') && ~isempty(env.bc_warm_eps)
        bcEps = round(env.bc_warm_eps);
    end
    % entW_bc kept soft (not 0) so the BC clone doesn't saturate the
    % logits; post-BC entropy anneal starts low (see 'es' below) so
    % fine-tuning doesn't re-randomize the clone.
    entW_bc = 0.03;

    for ep = 1:(bcEps + num_episodes)
        isBC = ep <= bcEps;
        epR  = ep - bcEps;
        if isBC
            entW = entW_bc;
        else
            if bcEps > 0, es = 0.01; else, es = ent_start; end
            entW = es + (ent_end-es)*(epR-1)/max(num_episodes-1,1);
        end
        sched = sampleSched_v5(env);
        [~, sOneHot] = schedContext_v5(sched, env);
        ctx = single(repmat(sOneHot, 1, N));
        [Strue, ~, meta] = generateChannel_v7(simTime, env);
        obsS = cqiFeedback_v7(Strue, env, meta.delays);

        burstObsM = zeros(simTime, N);
        for i = 1:N
            idx = max((1:simTime).' - meta.delays(i), 1);
            burstObsM(:, i) = double(meta.burst(idx));
        end

        aTgt = zeros(1, N);
        if isBC
            for i = 1:N
                [dc, vc] = userClass_v7(meta.delays(i), mean(meta.speeds(:,i)));
                [~, aTgt(i)] = min(abs(A - env.fixed_class_tab(dc, vc)));
            end
        end

        off = zeros(1, N); H = harqInit_v7(N); nk = zeros(1, N);
        prevRef = baseCQI_v5(obsS(1,:) + off, env);
        R_emas = zeros(1,N); ema = 0.05; ep_tp = 0; ep_ent = 0;

        for t = 1:simTime
            ref  = baseCQI_v5(obsS(t,:) + off, env);
            dRef = max(minD, min(maxD, ref - prevRef));
            dS   = obsS(t,:) - obsS(max(t-1,1),:);
            t0 = max(t-4, 1);
            slope4 = (obsS(t,:) - obsS(t0,:)) / max(t - t0, 1);
            S = stateFeat_v7(env.state_tier, ref, dRef, obsS(t,:), dS, off, ...
                             slope4, nk, H, meta.speeds(t,:), burstObsM(t,:), ctx);

            logits = extractdata(predict(actorNet, toDl(S)));
            logits = double(logits) - max(logits, [], 1);
            P = exp(logits) ./ sum(exp(logits), 1);
            ep_ent = ep_ent + mean(-sum(P.*log(P+1e-12), 1));

            aIdx = zeros(1,N); sent = zeros(1,N); pot = zeros(1,N);
            for i = 1:N
                if isBC
                    a = aTgt(i);
                else
                    a = find(rand <= cumsum(P(:,i)), 1, 'first');
```

```
                if isempty(a), a = nA; end
            end
            aIdx(i) = a;
            sent(i) = max(minC, min(maxC, ref(i) + A(a)));
            if env.harq_on && H.pend(i)
                pot(i) = tbsFromMCSandRB(H.mcs(i), env.total_RB) * 1e-3;
            else
                pot(i) = tbsFromMCSandRB(sent(i)-1, env.total_RB) * 1e-3;
            end
        end

        rb = schedule_users_v5(sched, t, N, R_emas, pot, env.total_RB);
        [tp_now, off, H, ~, nEv] = txStep_v7(sent, Strue(t,:), rb, off, H, env);
        schedM = rb > 0;
        nk(schedM) = 0.7*nk(schedM) + 0.3*double(nEv(schedM));
        nk(~schedM) = 0.95*nk(~schedM);
        R_emas = (1-ema)*R_emas + ema*tp_now; ep_tp = ep_tp + sum(tp_now);
        r_cell = sum(tp_now) / env.reward_scale;

        for i = 1:N
            nb = nb + 1; bufS(:,nb) = S(:,i); bufA(nb) = aIdx(i); bufR(nb) = single(r_cell);
        end
        prevRef = ref;

        if nb >= cap || t == simTime
            tn = min(t+1, simTime);
            refN  = baseCQI_v5(obsS(tn,:) + off, env);
            dRefN = max(minD, min(maxD, refN - ref));
            dSN = obsS(tn,:) - obsS(max(tn-1,1),:);
            t0n = max(tn-4, 1);
            slope4N = (obsS(tn,:) - obsS(t0n,:)) / max(tn - t0n, 1);
            sN = stateFeat_v7(env.state_tier, refN, dRefN, obsS(tn,:), dSN, off, ...
                              slope4N, nk, H, meta.speeds(tn,:), burstObsM(tn,:), ctx);
            Vb = extractdata(predict(criticNet, toDl(sN)));

            K = nb; Ts = K/N; rets = zeros(1,K,'single');
            for i = 1:N
                R = Vb(i);
                for tt = Ts:-1:1
                    idx = (tt-1)*N + i; R = bufR(idx) + gamma*R; rets(idx) = R;
                end
            end

            if isBC
                [aG, cG] = dlfeval(@bc_grads, actorNet, criticNet, ...
                                   toDl(bufS(:,1:K)), bufA(1:K), toDl(rets), entW);
            else
                [aG, cG] = dlfeval(@a2c_grads, actorNet, criticNet, ...
                                   toDl(bufS(:,1:K)), bufA(1:K), toDl(rets), entW);
            end
            aG = clipGlobal(aG, clip_norm); cG = clipGlobal(cG, clip_norm);
            aIt = aIt+1; [actorNet, aAvg, aAvgSq] = adamupdate(actorNet, aG, aAvg, aAvgSq, aIt, lr_actor);
            cIt = cIt+1; [criticNet, cAvg, cAvgSq] = adamupdate(criticNet, cG, cAvg, cAvgSq, cIt, lr_critic);
            nb = 0;
        end
    end
    if isBC
        if mod(ep, 25) == 0
            fprintf(' A2C v7 BC ep %3d/%d cellTp=%.2f H=%.3f\n', ...
                    ep, bcEps, ep_tp/simTime, ep_ent/simTime);
        end
    else
        learning_curve(epR) = ep_tp / simTime; entropy_curve(epR) = ep_ent / simTime;
        if mod(epR, 25) == 0
            fprintf(' A2C v7 ep %3d/%d cellTp=%.2f H=%.3f entW=%.3f\n', ...
                    epR, num_episodes, learning_curve(epR), entropy_curve(epR), entW);
        end
    end
end
```

```
end


function [aGrad, cGrad] = a2c_grads(actorNet, criticNet, S, Aacts, R, entW)
    V = forward(criticNet, S);
    adv = R - V;
    cLoss = mean(adv.^2);

    logits = forward(actorNet, S);
    m = max(logits, [], 1);
    logZ = m + log(sum(exp(logits - m), 1));
    logp_all = logits - logZ;
    B = size(S,2); An = size(logits,1);
    mask = zeros(An, B, 'like', extractdata(logits));
    for b = 1:B, mask(Aacts(b), b) = 1; end
    mask = dlarray(mask);
    logp = sum(logp_all .* mask, 1);

    advD = double(gather(extractdata(adv)));
    sd = std(advD);
    if sd > 1e-6, advD = (advD - mean(advD))/sd; else, advD = advD - mean(advD); end
    advD = dlarray(cast(advD, 'like', extractdata(logits)));

    p = exp(logp_all);
    entropy = -sum(p .* logp_all, 1);
    aLoss = -mean(logp .* advD) - entW*mean(entropy);

    cGrad = dlgradient(cLoss, criticNet.Learnables, 'RetainData', true);
    aGrad = dlgradient(aLoss, actorNet.Learnables);
end


function [aGrad, cGrad] = bc_grads(actorNet, criticNet, S, Atgt, R, entW)
% bc_grads Actor: cross-entropy on FixedPU's actions. Critic: MSE on
% the cloned rollout's observed n-step returns.
    V = forward(criticNet, S);
    cLoss = mean((R - V).^2);

    logits = forward(actorNet, S);
    m = max(logits, [], 1);
    logZ = m + log(sum(exp(logits - m), 1));
    logp_all = logits - logZ;
    B = size(S,2); An = size(logits,1);
    mask = zeros(An, B, 'like', extractdata(logits));
    for b = 1:B, mask(Atgt(b), b) = 1; end
    mask = dlarray(mask);
    ce = -mean(sum(logp_all .* mask, 1));

    p = exp(logp_all);
    entropy = -sum(p .* logp_all, 1);
    aLoss = ce - entW*mean(entropy);

    cGrad = dlgradient(cLoss, criticNet.Learnables, 'RetainData', true);
    aGrad = dlgradient(aLoss, actorNet.Learnables);
end


function grads = clipGlobal(grads, maxNorm)
    total = 0;
    for i = 1:height(grads)
        gg = double(gather(extractdata(grads.Value{i}))); total = total + sum(gg(:).^2);
    end
    nrm = sqrt(total);
    if nrm > maxNorm
        sc = maxNorm/nrm;
        for i = 1:height(grads), grads.Value{i} = grads.Value{i}*sc; end
    end
end
```

Listing 16: `DeepA2C_Shared_v7.m` — complete deep A2C trainer: network definition, rollout loop, n-step return

bootstrapping, the standard A2C gradient function, the behavior-cloning gradient function, and global gradient clipping.

Part IV The Comparator Suite

15 The Seven Schemes

Every campaign point scores the same seven schemes, all evaluated through the single shared harness `evalPolicy_v7.m`:

1. **Baseline** – fixed CQI mapping, *no* OLLA loop at all (the only scheme without it).
2. **OLLA** – the classical outer loop, RL action fixed at 0.
3. **Fixed** – OLLA plus a single best global constant offset, calibrated by grid search (§16).
4. **FixedPU** – OLLA plus a per-(delay-class,speed-class) constant offset, calibrated the same way but per class. This is the pre-registered primary comparator (§24): the strongest possible non-adaptive rule.
5. **QL v7** – tabular Q-learning (§12), Tier-A state.
6. **Tab AC v7** – tabular actor-critic (§13), Tier-A state.
7. **Deep A2C v7** – the deep learner (§14), state tier configurable.

16 Calibrating the Fixed-Offset Comparators

`calibrateFixedClass_v7.m` performs a pure grid search – no training, no gradients – over `env.fixed_cands` (default `-4:2`, widened from an original `-2:1` grid that was found to clip the true optimum; §**??**), scoring every candidate offset on a set of *calibration* traces drawn from a disjoint seed range (never the held-out evaluation traces used for the final scored comparison – this separation is what makes "oracle-calibrated" a fair, not circular, comparator). It records both the single best global offset and, per user class (§10, `userClass_v7`), the best offset for that class specifically.

```
function env = calibrateFixedClass_v7(env, nCalib, simTime)
% calibrateFixedClass_v7 Grid-search calibration of Fixed/FixedPU on
% held-out calibration traces (disjoint from evaluation traces).
    if nargin < 2 || isempty(nCalib), nCalib = 6; end
    if nargin < 3 || isempty(simTime), simTime = 200; end

    cands = env.fixed_cands; nC = numel(cands);
    acc = zeros(2, 3, nC); cnt = zeros(2, 3, nC);
    tot = zeros(1, nC);
    envE = env; envE.scheduler = 'pf';
    envE.olla_warm_ttis = 0;

    fprintf(' [calib] fixed-offset grid %s on %d calibration traces...\n', ...
            mat2str(cands), nCalib);
    for j = 1:nCalib
        rng(60000 + j);
        [Strue, ~, meta] = generateChannel_v7(simTime, env);
        obsS = cqiFeedback_v7(Strue, env, meta.delays);
        for c = 1:nC
            envE.fixed_offset = cands(c);
            rng(61000 + 100*j + c);
            tpc = evalPolicy_v7('fixed', [], Strue, obsS, envE, meta);
            b0 = min(env.burn_in, size(tpc,1)-1);
            pu = mean(tpc(b0+1:end,:), 1);
            tot(c) = tot(c) + sum(pu);
            for i = 1:env.N
                [dc, vc] = userClass_v7(meta.delays(i), mean(meta.speeds(:,i)));
                acc(dc, vc, c) = acc(dc, vc, c) + pu(i);
```

```
                cnt(dc, vc, c) = cnt(dc, vc, c) + 1;
            end
        end
    end

    [~, bestG] = max(tot);
    env.fixed_offset = cands(bestG);

    env.fixed_class_tab = zeros(2, 3);
    for dc = 1:2
        for vc = 1:3
            m = squeeze(acc(dc, vc, :) ./ max(cnt(dc, vc, :), 1));
            if all(cnt(dc, vc, :) == 0)
                env.fixed_class_tab(dc, vc) = env.fixed_offset;
            else
                [~, b] = max(m);
                env.fixed_class_tab(dc, vc) = cands(b);
            end
        end
    end

    fprintf(' [calib] best global fixed offset: %+d\n', env.fixed_offset);
    fprintf(' [calib] class table (rows: delay<=4, delay>=5 | cols: <10, 10-20, >20 m/s):\n');
    disp(env.fixed_class_tab);
end
```

Listing 17: `calibrateFixedClass_v7.m` — grid-search calibration of the Fixed and FixedPU comparators on held-out calibration traces.

17 The Unified Evaluation Harness

Every scheme, learned or fixed, is scored by exactly one function, parameterized by a `ptype` string. This is a deliberate design choice: it is impossible for one scheme to receive different physics, scheduler logic, or trace data than another, because they all pass through the same code path. The only optional feature is a "leakage-free warm-up" (`env.olla_warm_ttis`), used for one specific adversarial control (the `warm` campaign point, §27): it prepends an independent channel segment with the *same* user composition (same delays/speeds) but fresh channel randomness, letting OLLA's offset converge to steady state *before* scoring begins, without ever leaking the scored segment's own data into that warm-up.

```
function [tp, bler, cqi_err, act] = evalPolicy_v7(ptype, policy, SINR_true, obs_SINR, env, meta)
% evalPolicy_v7 Frozen-policy evaluation, shared by every scheme.
%   ptype: 'baseline' | 'olla' | 'fixed' | 'fixed_class' | 'ql' | 'ac' | 'a2c' | 'rand'
    [T, N] = size(SINR_true);
    A = env.actions; no_op = find(A == 0);
    maxC = env.max_CQI; minC = env.min_CQI; maxD = env.max_delta; minD = env.min_delta;
    SB = env.SINR_bins; te = env.trend_edges; OE = env.off_edges;
    useOLLA = ~strcmpi(ptype, 'baseline');

    % Leakage-free warm-up: independent channel segment, same users
    W = 0;
    burst = meta.burst; speeds = meta.speeds; delays = meta.delays;
    if useOLLA && isfield(env,'olla_warm_ttis') && env.olla_warm_ttis > 0
        W = round(env.olla_warm_ttis);
        rs = rng;
        rng(100000 + 7*W + 13*N + 101*round(mean(delays)));
        Sw = []; Ow = []; Bw = []; Vw = [];
        while size(Sw,1) < W
            [Sb, ~, mb] = generateChannel_v7(T, env, meta);
            Ob = cqiFeedback_v7(Sb, env, delays);
            Sw = [Sw; Sb]; Ow = [Ow; Ob]; %#ok<AGROW>
            Bw = [Bw; mb.burst]; Vw = [Vw; mb.speeds]; %#ok<AGROW>
        end
        rng(rs);
        SINR_true = [Sw(1:W,:); SINR_true];
        obs_SINR = [Ow(1:W,:); obs_SINR];
```

```
        burst = [Bw(1:W); burst];
        speeds = [Vw(1:W,:); speeds];
        T = T + W;
    end

    tp = zeros(T,N); bler = nan(T,N); cqi_err = zeros(T,N);
    act = no_op * ones(T, N);

    burstObsM = zeros(T, N);
    for i = 1:N
        idx = max((1:T).' - delays(i), 1);
        burstObsM(:, i) = double(burst(idx));
    end

    fco = zeros(1, N);
    if strcmpi(ptype, 'fixed_class')
        for i = 1:N
            [dc, vc] = userClass_v7(delays(i), mean(speeds(:,i)));
            fco(i) = env.fixed_class_tab(dc, vc);
        end
    end

    [sc, sOneHot] = schedContext_v5(env.scheduler, env);
    ctx = single(repmat(sOneHot, 1, N));
    isDeep = strcmpi(ptype, 'a2c');
    isTab = any(strcmpi(ptype, {'ql','ac'}));

    off = zeros(1, N); H = harqInit_v7(N);
    R_emas = zeros(1, N); ema = 0.05;
    nk = zeros(1, N);
    prevRef = baseCQI_v5(obs_SINR(1,:) + off, env);

    for t = 1:T
        ref  = baseCQI_v5(obs_SINR(t,:) + off, env);
        dRef = max(minD, min(maxD, ref - prevRef));
        dS   = obs_SINR(t,:) - obs_SINR(max(t-1,1),:);
        t0 = max(t-4, 1);
        slope4 = (obs_SINR(t,:) - obs_SINR(t0,:)) / max(t - t0, 1);

        if isDeep
            S = stateFeat_v7(env.state_tier, ref, dRef, obs_SINR(t,:), dS, off, ...
                             slope4, nk, H, speeds(t,:), burstObsM(t,:), ctx);
            logits = extractdata(predict(policy, dlarray(S, 'CB')));
        end

        sent = zeros(1,N); pot = zeros(1,N);
        for i = 1:N
            switch lower(ptype)
                case 'baseline'
                    cqi = ref(i);
                case 'olla'
                    cqi = ref(i);
                case 'fixed'
                    cqi = max(minC, min(maxC, ref(i) + env.fixed_offset));
                case 'fixed_class'
                    cqi = max(minC, min(maxC, ref(i) + fco(i)));
                case {'ql','ac'}
                    [~, sb] = min(abs(SB - obs_SINR(t,i)));
                    tbi = trendBin_v5(dS(i), te);
                    obn = 1 + sum(off(i) > OE);
                    tab = squeeze(policy(ref(i), dRef(i)+maxD+1, sb, tbi, obn, sc, :));
                    a = argmaxNoOp_v5(tab, no_op);
                    act(t,i) = a;
                    cqi = max(minC, min(maxC, ref(i) + A(a)));
                case 'a2c'
                    [~, a] = max(logits(:, i));
                    act(t,i) = a;
                    cqi = max(minC, min(maxC, ref(i) + A(a)));
                case 'rand'
```

```
                a = randi(numel(A));
                act(t,i) = a;
                cqi = max(minC, min(maxC, ref(i) + A(a)));
            otherwise
                error('evalPolicy_v7: unknown ptype "%s"', ptype);
        end
        sent(i) = cqi;
        if env.harq_on && H.pend(i)
            pot(i) = tbsFromMCSandRB(H.mcs(i), env.total_RB) * 1e-3;
        else
            pot(i) = tbsFromMCSandRB(sent(i)-1, env.total_RB) * 1e-3;
        end
        genie = max(1, min(15, find(SINR_true(t,i) >= env.CQI_thresholds, 1, 'last')));
        cqi_err(t,i) = abs(cqi - genie);
    end

    rb = schedule_users_v5(env.scheduler, t, N, R_emas, pot, env.total_RB);
    if useOLLA
        [tpr, off, H, bl1, nEv] = txStep_v7(sent, SINR_true(t,:), rb, off, H, env);
    else
        [tpr, ~, H, bl1, nEv] = txStep_v7(sent, SINR_true(t,:), rb, zeros(1,N), H, env);
    end
    tp(t,:) = tpr; bler(t,:) = bl1;
    sched = rb > 0;
    nk(sched) = 0.7*nk(sched) + 0.3*double(nEv(sched));
    nk(~sched) = 0.95*nk(~sched);
    R_emas = (1-ema)*R_emas + ema*tpr;
    prevRef = ref;
end

if W > 0
    tp = tp(W+1:end,:); bler = bler(W+1:end,:);
    cqi_err = cqi_err(W+1:end,:); act = act(W+1:end,:);
end
end
```

Listing 18: `evalPolicy_v7.m` — the single shared, frozen-policy evaluation function used for every scheme in every campaign point.

Part V The Full v1–v7 Design Evolution

This part documents, at the code and mechanism level, every major redesign between the first working version (v1) and the version whose results this report is about (v7). This history is not incidental: v6 (not v1) is the correct starting point for asking "does RL adaptivity have value here," precisely because four earlier, code-level bugs had to be found and fixed before that question could be asked honestly. The discipline established in this process – treat every surprising number, good or bad, as a clue to investigate, not a result to accept – is applied identically to the v7 campaign's own null result in Part VII.

18 v1: Absolute Per-User Actions and the Tie-Breaking Bug

The earliest version had agents choose an MCS directly and independently per user – no OLLA-relative encoding, no shared learning across users.

Symptom: trained tabular RL performed far below even the simplest fixed-mapping baseline – not a small gap, but a result consistent with the agent barely having trained at all.

Root cause (two compounding bugs):

1. MATLAB's `max()`, used to select the greedy action from a row of Q-values, breaks ties by returning the *first* matching index. Early in training, essentially all Q-values are tied at their common initial value (zero), so the "pick the best action" step was silently, systematically biased toward the lowest-index action every time a tie occurred, rather than an unbiased or randomized choice. A nominally neutral utility function was encoding a strong, unintended prior.

2. Each user had a fully separate Q-table, so learning experience was fragmented – no table benefited from another user's transitions, slowing convergence far below what pooling experience across users would achieve.

Fix: an explicit, deliberate tie-break rule biased toward the safe no-op action (this became `argmaxNoOp_v5.m`, Listing 12, reused unchanged through v7), and shared learning across users where architecturally appropriate.

19 v2–v3: The No-Op Imitation Trap

Subsequent iterations tried biasing training toward the do-nothing action and added auxiliary shaping rewards to encourage stability.

Symptom: the most heavily biased configurations produced the *worst* performance of anything tested – worse than doing nothing.

Root cause: in an offset-based action space where action 0 means "don't change the current estimate," biasing a policy toward the no-op action does not teach caution – it teaches the policy to freeze its estimate and stop tracking a channel that is, in reality, continuously changing. A no-op-biased agent stops adapting entirely, which is worse than either full adaptation or even fully random behavior.

Fix: the no-op bias and the shaping bonus were removed entirely.

General lesson carried forward: a bias intended to encode "prefer safe behavior" can instead encode "stop doing your job," if what looks safe does not actually mean what it appears to in context. Shaping rewards must be checked for what they operationally incentivize, not for how encouraging they sound.

20 v4: Baseline-Relative Action Rewrite

This version introduced a precursor to the OLLA-relative encoding (§8): actions were redefined as offsets from a fixed, static reference mapping, rather than absolute MCS choices from scratch.

Outcome: for the first time, a trained deep-learning policy edged out OLLA on raw throughput.

Concurrent fixes: the BLER waterfall curve was recalibrated to a more realistic shape; feedback delay and measurement noise were introduced into the CQI pipeline (the simulation had been unrealistically clean up to this point); and several standard deep-RL stability techniques were applied – correct value-estimate bootstrapping at the end of a finite episode, and standardization of the advantage signal used in the policy gradient (both well-known fixes for the high variance that plagues actor-critic training).

Lesson: relative action encoding, in this baseline-relative form, proved a significantly better-behaved formulation than absolute action selection – a lesson carried forward and later refined into the OLLA-relative form used from v6 onward.

21 v5: An "Impossible" Result Reveals Two Deeper Bugs

Before believing v4's promising result, it was re-audited.

Symptom: OLLA was measured losing to a naive baseline by over 160% – a number that is physically impossible for a control loop that, by its own design, can only move its offset by small increments per step. An implausibly large positive result deserves exactly the same scrutiny as a bad one – it usually means a bug in the *measurement pipeline*, not a real effect.

Root cause 1 – scheduler information leak: a wiring error let the resource-scheduling logic see a transmission attempt's *true, future* outcome when making a scheduling decision, rather than only the information genuinely available at scheduling time. This silently favored whichever policy happened to be more aggressive, independent of whether that aggression was actually a good decision – a lookahead-bias analogue of a backtested trading strategy peeking at future prices.

Root cause 2 – a reward "prisoner's dilemma": rewards were computed per-user rather than pooled at the cell level, creating a perverse incentive: a user (or the policy acting for it) could over-report its own channel quality to win more scheduling opportunities, even though this also raised its own risk of transmission failure. A direct instance of a multi-agent incentive-misalignment problem – a reward structure that rewards individually selfish behavior at the expense of the group outcome, even though every agent shares the same underlying resource.

Fix: the scheduler was restricted to only the information genuinely available at decision time (*reported*, never true, channel quality – see §7, `env.pf_metric = 'reported'`), and rewards were changed to a single, pooled, cooperative cell-level signal shared by every user's learning update – removing the incentive to misreport.

Lesson: an implausibly *good* result deserves exactly the same forensic scrutiny as an implausibly bad one. Scheduler information leakage and misaligned multi-agent incentives are recurring failure modes worth actively auditing for in any multi-user RL system, not just this one.

22 v6: OLLA-Relative Encoding Closes the Information Gap

With the v5 scheduler and reward bugs fixed, the honest result was that RL only *matched* OLLA, rather than beating it.

Diagnosis: OLLA has access to fast, per-transmission ACK/NACK feedback (§6) that was not being exposed to the RL agent's own observed state at all in the v5 setup. This is an informational asymmetry, not a real capability gap – OLLA was not smarter, it simply knew something the learner was never told.

Fix: the OLLA-relative action encoding described in §8 – give every learner (and every comparator built afterward) the identical internal OLLA loop, define the RL action as a correction on top of it, and expose the current OLLA offset as part of the observed state.

Outcome: v6 finally produced a genuinely fair, apples-to-apples comparison – and, as detailed next, even under this fair comparison its apparent success turned out to be incomplete once one more control was added.

22.1 The v6 Lesson: The Constant-Offset Control

Good experimental design does not just ask "does my sophisticated method beat a naive baseline" – it asks "does my sophisticated method beat the simplest thing I can think of." The relevant "simple thing" here is a **constant**: instead of a neural network that observes state and decides, pick a single, unchanging correction offset, calibrated in advance on separate data, and apply it identically forever regardless of what the environment does. A constant cannot, by definition, depend on state, adapt, or predict.

When this control was added to v6's evaluation, it matched the neural network's throughput almost exactly. A policy that cannot see, think, or adapt achieved the same performance as one specifically built to do all three. This is the finding that motivated the entire v7 redesign: if a constant is just as good, then whatever the neural network's advantage over OLLA consists of, it is fully explained by *where the operating point was placed* (running a bit hotter than OLLA's own default equilibrium) rather than by state-conditional adaptivity. This is termed a **setpoint effect** throughout this report: a measured gain that is really a single scalar shift, not evidence of adaptive decision-making.

23 v7: The Five Design Levers, Restated as a Rationale

Diagnosing *why* v6's environment let a constant be sufficient identified four specific environment simplifications, each removing a possible source of exploitable state-conditional value. v7 removes all four simultaneously, and adds one experimental (non-physical) control, for five levers total (D1–D5, cross-referenced to their defining sections above):

Lever	What it changes	Why it matters for adaptivity value
D1	HARQ with chase combining (§5)	Makes error genuinely costly; kills the "just run hot" degenerate optimum.
D2	Per-user heterogeneous delay/speed (§3, §2)	A single constant cannot fit every user simultaneously.
D3	Interference bursts + mid-trace speed switches (§2.3)	Creates genuine regime transitions a reactive/static rule structurally lags.
D4	Steep per-MCS BLER curves + Jakes fading (§4, §2.2)	Restores a real MCS-precision trade-off; couples speed to predictability.
D5	Two-tier state ablation (§10)	Isolates whether any RL result is explained by information access, not adaptivity.

```
function env = defaultEnv_v7(caseId, N)
% defaultEnv_v7 v7 MDP environment defaults.
    if nargin < 1 || isempty(caseId), caseId = 23; end
    if nargin < 2 || isempty(N),   N = 10;     end

    % Radio / channel
    env.fc         = 2e9;
    env.hBS        = 25;
    env.hUE        = 1.5;
    env.txPower_dBm = 43;
    env.G_dBi      = 8;
    env.O2I_dB     = 20;
    env.IM_dB      = 6;
    env.NF_dB      = 9;
    env.BW         = 20e6;
    env.sigma_SF   = 6;
    env.d_corr     = 50;
    env.total_RB   = 100;
    env.T_tti      = 1e-3;

    % Fading (Jakes AR(1) per user)
    env.jakes = 1;
    env.rho_fading = 0.95;

    % Heterogeneous users
```

```
    env.hetero   = 1;
    env.speed_min = 3;
    env.speed_max = 30;
    env.delay_set = 2:8;
    env.delay_hom = 4;

    % Non-stationarity
    env.burst_on      = 1;
    env.burst_IM_dB   = 9;
    env.burst_dwell_calm = 250;
    env.burst_dwell_burst = 120;
    env.speed_switch  = 1;

    % HARQ
    env.harq_on     = 1;
    env.harq_max_retx = 3;
    env.harq_comb_dB = 2.5;

    % CQI feedback
    env.cqi_delay = 4;
    env.sigma_meas = 1.0;

    % MDP
    env.bc_warm_eps = 0;
    env.gamma      = 0.95;
    env.reward_scale = 30;
    env.reward_mode = 'throughput';
    env.reward_agg = 'cell';

    % CQI mapping (anchored at BLER=0.1)
    env.CQI_thresholds = [-Inf, -6.7, -5.0, -3.2, -1.5, 0.8, 2.0, 4.5, 6.8, ...
                          9.2, 11.5, 14.0, 16.3, 18.7, 20.5, 21.8];

    % State discretization
    env.SINR_bins = linspace(-10, 30, 20);
    env.max_CQI = 15; env.min_CQI = 1;
    env.max_delta = 5; env.min_delta = -5;
    env.actions = -3:3;
    env.off_edges = [-3, -1, 1, 3];
    env.trend_edges = [-1, 1];
    env.n_trend = 3;

    % State tier
    env.state_tier = 'B';

    % Steady-state scoring
    env.burn_in = 50;

    % OLLA
    env.olla_target    = 0.1;
    env.olla_step_down = 0.5;
    env.olla_warm_passes = 0;
    env.olla_warm_ttis = 0;

    % Fixed-offset comparators
    env.fixed_offset  = -1;
    env.fixed_class_tab = -1*ones(2,3);
    env.fixed_cands   = -4:2;

    % Scheduler
    env.pf_metric = 'reported';
    env.N = N; env.scheduler = 'pf'; env.mobility = 'rwp';
    env.areaSize = 1000; env.bsPos = [500, 500]; env.speed = 5;
    env.sched_list        = {'pf', 'rr', 'eq'};
    env.train_schedulers  = {'pf', 'rr', 'eq'};
    env.train_sched_weights = [0.50, 0.25, 0.25];
    env.caseId = caseId;
    env.use_gpu = false;
end
```

Listing 19: `defaultEnv_v7.m` — the single source of truth for the entire v7 MDP: every physical constant, every design-lever parameter, every algorithm hyperparameter default.

Part VI Experimental Protocol

24 Pre-Registration

Before running the campaign, the design was committed in writing: exactly what would count as success, decided before seeing results. This practice, borrowed from clinical trials, guards against a well-known failure mode – if success criteria can be chosen *after* seeing the data, some metric, subset, or scheduler will almost always be found under which the preferred method looks good, purely by chance (the statistical multiple-comparisons problem: the more ways data is allowed to be cut in search of significance, the higher the chance of finding spurious significance somewhere).

Pre-registered win condition: the neural-network RL policy must beat the best-tuned, per-user-class-calibrated constant offset (FixedPU), under the industry-standard Proportional Fair scheduler, at significance $p < 0.05$ on a paired statistical test.

Each clause has a specific rationale:

- **Comparator is a constant, not just OLLA** – §22.1 established that beating OLLA alone does not distinguish "learned something clever" from "found a good setpoint."
- **Per-class (FixedPU), not global (Fixed)** – §23 (D2): a single global constant is provably suboptimal under heterogeneity, so comparing against a per-class constant closes the loophole of "RL wins just by learning to treat different users differently," something a lookup table keyed on user identity could also do with no real state-conditioning.
- **A specific scheduler (PF)** – the scheduler actually used in deployed networks; a result true only under an unrealistic scheduling rule would have limited relevance.
- **Paired test at $p < 0.05$** – "paired" means every scheme is scored on *exactly* the same 30 held-out test traces (never seen during training or calibration), removing noise that would otherwise have to be averaged away. The specific test, the Wilcoxon signed-rank test, is a nonparametric alternative to a paired t-test that does not assume the compared differences are normally distributed – appropriate here since throughput is bounded and often skewed.

25 The Comparison Harness

`runComparison_v7.m` is the single entry point that executes one full experimental point: calibrates the fixed-offset comparators (Phase 0), trains all three learners across `n_seeds` independent random seeds (Phase 1), evaluates all seven schemes on `n_eval` held-out traces identical across schemes and across every evaluation scheduler (Phase 2), and computes and saves all metrics plus the pre-registered paired statistical tests (Phase 3).

```
function results = runComparison_v7(opts)
% runComparison_v7 Comparison harness: calibrate, train, evaluate, report.
    if nargin < 1, opts = struct(); end
    g = @(f,d) get_def(opts, f, d);

    Nopt     = g('N', 10);
    simTime = g('simTime', 200);
    ep_tab  = g('ep_tab', 300);
    ep_deep = g('ep_deep', 300);
    n_seeds = g('n_seeds', 5);
    n_eval  = g('n_eval', 30);
    baseSeed = g('base_seed', 42);
    traceSeedBase = g('trace_seed_base', 9000);
    nCalib  = g('n_calib', 6);

```

```
here = fileparts(mfilename('fullpath'));
addpath(fullfile(here, '..', 'v5'));
addpath(fullfile(here, '..', 'case2_3'));

env = defaultEnv_v7(23, Nopt);
for f = {'hetero','burst_on','burst_IM_dB','burst_dwell_calm', ...
        'burst_dwell_burst','speed_switch','harq_on','harq_max_retx', ...
        'harq_comb_dB','state_tier','olla_warm_ttis','actions', ...
        'sigma_meas','speed','cqi_delay','delay_hom','use_gpu', ...
        'train_sched_weights','bc_warm_eps','fixed_cands'}
    if isfield(opts, f{1}) && ~isempty(opts.(f{1})), env.(f{1}) = opts.(f{1}); end
end
eval_scheds = g('eval_schedulers', {'pf','rr','eq'});

fprintf('=========================================================================\n');
fprintf(' v7 comparison | N=%d hetero=%.2f burst=%d(+%g dB) harq=%d tier=%s\n', ...
       env.N, env.hetero, env.burst_on, env.burst_IM_dB, env.harq_on, env.state_tier);
fprintf(' seeds=%d traces=%d ep_tab=%d ep_deep=%d simTime=%d warm_ttis=%d\n', ...
       n_seeds, n_eval, ep_tab, ep_deep, simTime, env.olla_warm_ttis);
fprintf(' started %s\n', datestr(now,'yyyy-mm-dd HH:MM:SS'));
fprintf('=========================================================================\n');

% Phase 0: calibrate fixed-offset comparators
env = calibrateFixedClass_v7(env, nCalib, simTime);

% Phase 1: training
fprintf('PHASE 1 -- Training (%d seeds x 3 algorithms)\n', n_seeds);
QLs = cell(1,n_seeds); ACs = cell(1,n_seeds); A2Cs = cell(1,n_seeds);
lc_ql = zeros(ep_tab, n_seeds); lc_ac = zeros(ep_tab, n_seeds);
lc_dac = zeros(ep_deep, n_seeds); ent_dac = zeros(ep_deep, n_seeds);
for s = 1:n_seeds
    fprintf(' seed %d/%d (%s)\n', s, n_seeds, datestr(now,'HH:MM:SS'));
    t0 = tic;
    [QLs{s}, lc_ql(:,s)] = QL_Shared_v7(simTime, ep_tab, baseSeed + s, env);
    fprintf('   [Q-Learn ] %7.1f s final cellTp = %5.2f\n', toc(t0), lc_ql(end,s));
    t0 = tic;
    [ACs{s}, ~, lc_ac(:,s)] = AC_Shared_v7(simTime, ep_tab, baseSeed + 100 + s, env);
    fprintf('   [Tab AC ] %7.1f s  final cellTp = %5.2f\n', toc(t0), lc_ac(end,s));
    t0 = tic;
    [A2Cs{s}, ~, lc_dac(:,s), ent_dac(:,s)] = ...
        DeepA2C_Shared_v7(simTime, ep_deep, baseSeed + 200 + s, env);
    fprintf('   [Deep A2C] %7.1f s final cellTp = %5.2f\n', toc(t0), lc_dac(end,s));
end

% Phase 2: held-out traces, identical for every scheme
fprintf('PHASE 2 -- Evaluation on %d held-out traces\n', n_eval);
traces = struct('SINR',[], 'obsS',[], 'meta',[]); traces(n_eval).SINR = [];
for j = 1:n_eval
    rng(traceSeedBase + j);
    [Strue, ~, meta] = generateChannel_v7(simTime, env);
    obsS = cqiFeedback_v7(Strue, env, meta.delays);
    traces(j).SINR = Strue; traces(j).obsS = obsS; traces(j).meta = meta;
end

schemes = {'Baseline','OLLA','Fixed','FixedPU','QL v7','Tab AC v7','Deep A2C v7'};
ptypes = {'baseline','olla','fixed','fixed_class','ql','ac','a2c'};
K = numel(schemes);
detK = 4;

for sc = 1:numel(eval_scheds)
    envE = env; envE.scheduler = eval_scheds{sc};
    M = init_metrics(K, n_seeds, n_eval);
    for j = 1:n_eval
        mt = traces(j).meta;
        for k = 1:K
            if k <= detK, ns = 1; else, ns = n_seeds; end
            for s = 1:ns
                pol = [];
                if k==5, pol=QLs{s}; elseif k==6, pol=ACs{s}; elseif k==7, pol=A2Cs{s}; end
```

```
                rng(77000 + 100*j + 10*k + s);
                [tp, bl, ce] = evalPolicy_v7(ptypes{k}, pol, ...
                                traces(j).SINR, traces(j).obsS, envE, mt);
                b0 = min(env.burn_in, size(tp,1)-1);
                tpB = tp(b0+1:end,:); blB = bl(b0+1:end,:); ceB = ce(b0+1:end,:);
                bu  = mt.burst(b0+1:end);
                pu = mean(tpB, 1);
                M.cell_tp{k}(s,j) = sum(pu);
                M.edge_tp{k}(s,j) = min(pu);
                ok = ~isnan(blB);
                if any(ok(:)), M.bler{k}(s,j) = mean(blB(ok)); else, M.bler{k}(s,j) = NaN; end
                M.jain{k}(s,j) = sum(pu)^2 / (env.N * max(sum(pu.^2), eps));
                M.cqie{k}(s,j) = mean(ceB(:));
                if any(bu), M.tp_burst{k}(s,j) = sum(mean(tpB(bu,:), 1));
                else,       M.tp_burst{k}(s,j) = NaN; end
                if any(~bu), M.tp_calm{k}(s,j) = sum(mean(tpB(~bu,:), 1));
                else,       M.tp_calm{k}(s,j) = NaN; end
            end
        end
    end
    allM(sc) = M; %#ok<AGROW>
end

% Phase 3: report
txt = header_txt(env, n_seeds, n_eval, ep_tab, ep_deep, simTime, traceSeedBase, baseSeed);
pv = struct();
for sc = 1:numel(eval_scheds)
    [tsec, pv_sc] = render_block(allM(sc), upper(eval_scheds{sc}), schemes, K, n_seeds, n_eval);
    txt = [txt tsec]; %#ok<AGROW>
    if sc == 1, pv = pv_sc; end
end

results.env = env; results.schemes = schemes; results.metrics = allM;
results.eval_schedulers = eval_scheds; results.pvals = pv;
results.learning = struct('ql',lc_ql,'ac',lc_ac,'dac',lc_dac,'ent',ent_dac);
results.policies = struct('QL',{QLs},'AC',{ACs},'A2C',{A2Cs});
results.opts = opts;

tag = g('tag', 'run');
outdir = fullfile(here, 'Dataset');
if ~exist(outdir, 'dir'), mkdir(outdir); end
mat_path = fullfile(outdir, sprintf('results_v7_%s.mat', tag));
txt_path = fullfile(outdir, sprintf('results_v7_%s.txt', tag));
if exist(mat_path, 'file') || exist(txt_path, 'file')
    stamp = datestr(now, 'yyyymmdd_HHMMSS');
    mat_path = fullfile(outdir, sprintf('results_v7_%s_%s.mat', tag, stamp));
    txt_path = fullfile(outdir, sprintf('results_v7_%s_%s.txt', tag, stamp));
end
save(mat_path, 'results', '-v7.3');
fid = fopen(txt_path, 'w'); fprintf(fid, '%s', txt); fclose(fid);
fprintf(' Saved:\n   %s\n   %s\n', mat_path, txt_path);
end


function v = get_def(s,f,d), if isfield(s,f)&&~isempty(s.(f)), v=s.(f); else, v=d; end, end

function M = init_metrics(K, n_seeds, n_eval)
    for k = 1:K
        if k <= 4, ns = 1; else, ns = n_seeds; end
        M.cell_tp{k}=zeros(ns,n_eval); M.edge_tp{k}=zeros(ns,n_eval);
        M.bler{k}=zeros(ns,n_eval); M.jain{k}=zeros(ns,n_eval); M.cqie{k}=zeros(ns,n_eval);
        M.tp_burst{k}=nan(ns,n_eval); M.tp_calm{k}=nan(ns,n_eval);
    end
end

function txt = header_txt(env, n_seeds, n_eval, ep_tab, ep_deep, simTime, tsb, bs)
    txt = sprintf('================================================================\n');
    txt = [txt sprintf('v7 RESULTS N=%d hetero=%.2f burst_on=%d(+%g dB, dwell %d/%d)\n', ...
        env.N, env.hetero, env.burst_on, env.burst_IM_dB, env.burst_dwell_calm, env.burst_dwell_burst)];
```

```
    txt = [txt sprintf('harq_on=%d(max %d, comb %.1f dB) jakes=%d speed_switch=%d tier=%s\n', ...
        env.harq_on, env.harq_max_retx, env.harq_comb_dB, env.jakes, env.speed_switch, env.state_tier)];
    txt = [txt sprintf('seeds=%d eval_traces=%d ep_tab=%d ep_deep=%d simTime=%d\n', ...
        n_seeds, n_eval, ep_tab, ep_deep, simTime)];
    txt = [txt sprintf('BW=%g MHz RB=%d sigma_meas=%.2f dB actions=%s\n', ...
        env.BW/1e6, env.total_RB, env.sigma_meas, mat2str(env.actions))];
    txt = [txt sprintf('olla: target=%.2f step=%.2f warm_ttis=%d fixed=%+d class_tab=%s\n', ...
        env.olla_target, env.olla_step_down, env.olla_warm_ttis, env.fixed_offset, ...
        mat2str(env.fixed_class_tab))];
    bce = 0; if isfield(env,'bc_warm_eps'), bce = env.bc_warm_eps; end
    txt = [txt sprintf('train_sched_weights=%s bc_warm_eps=%d fixed_cands=%s\n', ...
        mat2str(env.train_sched_weights, 2), bce, mat2str(env.fixed_cands))];
    txt = [txt sprintf('base_seed=%d trace_seed_base=%d saved %s\n', ...
        bs, tsb, datestr(now,'yyyy-mm-dd HH:MM:SS'))];
    txt = [txt sprintf('================================================================\n\n')];
    fprintf('%s', txt);
end

function [txt, pv] = render_block(M, lbl, schemes, K, n_seeds, n_eval)
    txt = sprintf('---- scheduler %s ----\n', lbl);
    tr = cell(1,K);
    for k = 1:K, tr{k} = mean(M.cell_tp{k}, 1); end
    mm = cellfun(@mean, tr);
    ci = cellfun(@(x) 1.96*std(x)/sqrt(numel(x)), tr);
    edge = zeros(1,K); bler = zeros(1,K); jain = zeros(1,K);
    tb = zeros(1,K); tc = zeros(1,K);
    for k = 1:K
        edge(k) = mean(mean(M.edge_tp{k},1));
        bler(k) = mean(M.bler{k}(:), 'omitnan');
        jain(k) = mean(M.jain{k}(:));
        tb(k)   = mean(M.tp_burst{k}(:), 'omitnan');
        tc(k)   = mean(M.tp_calm{k}(:), 'omitnan');
    end
    base = mm(1);
    [~, bi] = max(mm);

    hdr = sprintf(' %-12s | %16s | %8s | %7s | %7s | %6s | burst / calm Tp\n', ...
                  'Scheme','Cell Tp (Mbps)','d vs BL','EdgeTp','BLER1','Jain');
    sep = [' ', repmat('-', 1, 86), newline];
    txt = [txt hdr sep]; fprintf('%s%s', hdr, sep);
    for k = 1:K
        if k == 1, dstr = ' (ref)'; else, dstr = sprintf('%+6.1f%%', 100*(mm(k)-base)/max(base,eps)); end
        mk = ' '; if k == bi, mk = '*'; end
        line = sprintf(' %-12s |%c%7.2f +/- %4.2f | %8s | %7.2f | %7.4f | %6.3f | %5.2f / %5.2f', ...
            schemes{k}, mk, mm(k), ci(k), dstr, edge(k), bler(k), jain(k), tb(k), tc(k));
        fprintf('%s\n', line); txt = [txt line newline]; %#ok<AGROW>
        if k == 4, fprintf('%s', sep); txt = [txt sep]; end %#ok<AGROW>
    end
    note = sprintf([' (CI: trace-level n=%d; BLER1 = first-transmission BLER;\n' ...
                    '  burst/calm = cell Tp during interference bursts vs calm)\n'], n_eval);
    fprintf('%s', note); txt = [txt note];

    pv = struct();
    if exist('signrank','file') == 2
        refs = [2 3 4 6 1];
        names = {'OLLA ','Fixed ','FixedPU','TabAC ','Baseline'};
        fields = {'a2c_vs_olla','a2c_vs_fixed','a2c_vs_fixedPU','a2c_vs_tabac','a2c_vs_base'};
        line = sprintf(' Wilcoxon (paired, n=%d traces), margin = A2C vs comparator:\n', n_eval);
        for q = 1:numel(refs)
            p = signrank(tr{7}, tr{refs(q)});
            pv.(fields{q}) = p;
            mrg = 100*(mean(tr{7}) - mean(tr{refs(q)})) / max(mean(tr{refs(q)}), eps);
            if mrg >= 0, dirw = 'WIN '; else, dirw = 'LOSS'; end
            note2 = ''; if q == 3, note2 = ' <-- pre-registered win condition'; end
            line = [line sprintf('  A2C vs %s %+6.1f%% p=%.4g %s [%s]%s\n', ...
                   names{q}, mrg, p, stars(p), dirw, note2)]; %#ok<AGROW>
        end
        fprintf('%s\n', line); txt = [txt line newline];
    end
```

```
end

function s = stars(p)
    if     p < 0.001, s = '***';
    elseif p < 0.01, s = '**';
    elseif p < 0.05, s = '*';
    else,            s = '(n.s.)';
    end
end
```

Listing 20: `runComparison_v7.m` — the complete four-phase experimental harness: calibration, training, held-out evaluation, and statistical reporting.

26 Point Presets and Campaign Design

`run_v7_point.m` wraps the harness in a resumable, logged, one-preset-per-process runner (a fresh MATLAB process per point, since a lesson from v6 was that long single-session sweeps degrade performance roughly 20× and can stall). Every preset used in the campaign, and its exact configuration, is defined here.

```
function run_v7_point(preset, val)
% run_v7_point Run one v7 experiment, headless, resumable, logged.
%   matlab -batch "run_v7_point('headline')"
    if nargin < 1 || isempty(preset), preset = 'smoke'; end
    if nargin < 2, val = []; end

    here = fileparts(mfilename('fullpath'));
    cd(here);
    addpath(fullfile(here, '..', 'v5'));
    addpath(fullfile(here, '..', 'case2_3'));
    if ~exist(fullfile(here,'logs'), 'dir'), mkdir(fullfile(here,'logs')); end
    logfile = fullfile(here, 'logs', 'v7_master_log.txt');
    diary(logfile); diary on;

    o = struct('n_seeds',5, 'n_eval',30, 'ep_tab',300, 'ep_deep',500);
    switch lower(preset)
        case 'smoke'
            o.n_seeds = 1; o.n_eval = 3; o.ep_tab = 40; o.ep_deep = 40;
            o.n_calib = 2; o.tag = 'smoke';
        case 'phase0'
            o.hetero = 0; o.burst_on = 0; o.speed_switch = 0;
            o.tag = 'phase0_calm';
        case 'hetero'
            if isempty(val), val = 1; end
            o.hetero = val; o.burst_on = 0; o.speed_switch = 1;
            o.tag = sprintf('hetero%03d', round(100*val));
        case 'burst'
            if isempty(val), val = 9; end
            o.hetero = 1; o.burst_on = 1; o.burst_IM_dB = val; o.speed_switch = 1;
            o.tag = sprintf('burst%02ddB', round(val));
        case 'headline'
            o.hetero = 1; o.burst_on = 1; o.burst_IM_dB = 9; o.speed_switch = 1;
            o.tag = 'headline';
        case 'tiera'
            o.hetero = 1; o.burst_on = 1; o.burst_IM_dB = 9; o.speed_switch = 1;
            o.state_tier = 'A'; o.tag = 'headline_tierA';
        case 'warm'
            o.hetero = 1; o.burst_on = 1; o.burst_IM_dB = 9; o.speed_switch = 1;
            o.olla_warm_ttis = 1000; o.tag = 'headline_warm1000';
        case 'replicate'
            o.hetero = 1; o.burst_on = 1; o.burst_IM_dB = 9; o.speed_switch = 1;
            o.base_seed = 10042; o.trace_seed_base = 19000; o.tag = 'headline_fresh';
        case 'tier1'
            o.hetero = 1; o.burst_on = 1; o.burst_IM_dB = 9; o.speed_switch = 1;
            o.train_sched_weights = [1 0 0];
            o.bc_warm_eps = 150;
            o.tag = 'headline_tier1';
```

```
        case 'tier1nobc'
            o.hetero = 1; o.burst_on = 1; o.burst_IM_dB = 9; o.speed_switch = 1;
            o.train_sched_weights = [1 0 0];
            o.bc_warm_eps = 0;
            o.actions = -5:3;
            o.tag = 'headline_tier1_nobc';
        case 'tier1b'
            o.hetero = 1; o.burst_on = 1; o.burst_IM_dB = 9; o.speed_switch = 1;
            o.train_sched_weights = [1 0 0];
            o.bc_warm_eps = 150;
            o.actions = -5:3;
            o.tag = 'headline_tier1b';
        case 'tier1rep'
            o.hetero = 1; o.burst_on = 1; o.burst_IM_dB = 9; o.speed_switch = 1;
            o.train_sched_weights = [1 0 0];
            o.bc_warm_eps = 150;
            o.actions = -5:3;
            o.base_seed = 10042; o.trace_seed_base = 19000;
            o.tag = 'headline_tier1b_fresh';
        otherwise
            error('run_v7_point: unknown preset "%s"', preset);
    end

    fprintf('\n##### v7 POINT "%s" [pid %d] started %s #####\n', ...
            o.tag, feature('getpid'), datestr(now,'yyyy-mm-dd HH:MM:SS'));

    hit = dir(fullfile(here, 'Dataset', sprintf('results_v7_%s.mat', o.tag)));
    if ~isempty(hit)
        fprintf('##### SKIP: results_v7_%s.mat exists - point already complete #####\n', o.tag);
        diary off; return;
    end

    t0 = tic;
    try
        runComparison_v7(o);
        fprintf('##### v7 POINT "%s" COMPLETE (%.2f h) #####\n', o.tag, toc(t0)/3600);
    catch ME
        fprintf(2, '\n!!! v7 POINT "%s" FAILED after %.2f h: %s\n%s\n', ...
                o.tag, toc(t0)/3600, ME.identifier, getReport(ME, 'extended'));
    end
    diary off;
end
```

Listing 21: `run_v7_point.m` — every campaign-point preset (`phase0`, `hetero`, `burst`, `headline`, `tierA`, `warm`, `replicate`, `tier1`, `tier1nobc`, `tier1b`, `tier1rep`) with inline rationale for each parameter choice.

Part VII Results

27 The Nine-Point Pre-Registered Campaign

Table 1 reports every pre-registered campaign point, PF scheduler, at full precision as recorded by the harness (see also the shorter summary table in `v7_comprehensive_evaluation_report.pdf`).

Table 1: Full v7 Campaign Results, PF Scheduler ($N = 10$, 5 seeds, 30 held-out traces per point)

Point (tag)	Baseline	OLLA	Fixed	FixedPU	QL	TabAC	A2C	vs. FixedPU (p)	vs. OLLA (p)
`phase0_calm` (hetero=0, no burst)	4.14	4.48	4.79	4.68	3.53	3.64	4.59	−1.8% (0.280)	+2.5% (0.271)
`hetero050` (hetero=0.5)	3.80	4.28	4.67	4.59	3.71	3.58	4.40	−4.2% (0.026*)	+2.7% (0.319)
`hetero100` (hetero=1.0, no burst)	3.71	4.02	4.32	4.15	3.48	3.45	4.19	+0.9% (0.861)	+4.4% (0.141)
`burst06dB`	3.17	3.56	3.97	3.85	3.19	3.11	3.59	−6.9% (0.003**)	+0.8% (0.910)
`burst09dB` **(headline, vanilla)**	**3.10**	**3.29**	**3.79**	**3.66**	**3.04**	**3.01**	**3.63**	−0.7% **(0.517)**	+10.4% **(0.0006***)**
`burst12dB`	3.01	3.21	3.69	3.48	2.89	2.86	3.32	−4.5% (0.047*)	+3.4% (0.393)
`headline_tierA`	3.10	3.29	3.79	3.66	3.04	3.01	3.66	+0.1% (0.558)	+11.2% (0.0008***)
`headline_warm1000`	3.10	3.69	3.45	3.48	3.45	3.15	3.47	−0.3% (0.926)	−5.9% (0.0003***)
`headline_fresh` (replicate, seed 10042)	3.60	3.81	4.08	4.04	3.30	3.25	3.86	−4.4% (0.006**)	+1.3% (0.614)

Result: 0 of 9 pre-registered tests passed. Deep A2C never significantly beats FixedPU on PF; three of nine points show a significant *loss*. A2C beats OLLA at 6 of 9 points (significant at 2), and never loses to OLLA significantly except at `warm1000` – the deliberately handicapping control where OLLA is given a 1000-TTI head start to converge before scoring, which is itself informative (§22.1: OLLA converged to a better setpoint than its own cold-start default, closing most of the setpoint gap).

28 Diagnosis: Five Root Causes for the Zero-Win Result

Two very different explanations are consistent with a 0/9 result: (a) there is genuinely no exploitable adaptivity value in this class of problem, and the honest conclusion is a well-supported negative result; or (b) something in the experimental *pipeline itself* – a bug, a mismatch, a hidden unfair comparison – is preventing the RL agent from reaching its true achievable performance, and the null is an artifact of that flaw. These look identical from the outside (both produce "0 of 9"), but only one is a trustworthy scientific finding. Five concrete issues were identified by direct code inspection, not speculation.

28.1 Root Cause 1: Calibration/Training Distribution Asymmetry

`calibrateFixedClass_v7.m` (Listing 17) tunes FixedPU exclusively on the PF scheduler (`envE.scheduler = 'pf'` is hard-set inside the calibration loop). But the vanilla Deep A2C trainer sampled its training scheduler from `env.train_schedulers` with weights `env.train_sched_weights = [0.50, 0.25, 0.25]` across PF/RR/EQ (`sampleSched_v5.m`, Listing 8) – meaning only half of A2C's training experience was on the scheduler the pre-registered test actually measures. The comparator was specialized for the test condition; the learner was not.

28.2 Root Cause 2: Action-Set / Calibration-Grid Clipping

`env.fixed_cands` was originally `-2:1` (4 candidates). The selected best global offset sat at -2 – the edge of the searched range, a classic optimization warning sign that the true optimum lies outside the searched window. After widening the grid, one user class's true per-class optimum came out at -4, entirely outside Deep A2C's contemporaneous action set of `-3:3`. The comparator's true best answer was, literally, an option the learner was never given.

28.3 Root Cause 3: Undertraining on Cold Start

With 500 training episodes and no informed initialization, vanilla Deep A2C settled roughly 5% *below* a policy it can trivially represent exactly (recall: action 0 makes it behave identically to OLLA, and a constant-action policy is a simple special case of what a small MLP can express). A model failing to match a policy well within its own expressive capacity is a symptom of an optimization problem (poor initialization, insufficient training, weak exploration), not evidence about the underlying task's difficulty.

28.4 Root Cause 4: Entropy Restart After Behavior Cloning

A first attempt to fix Root Cause 3 introduced a 150-episode behavior-cloning (BC) warm start (§14.1) from FixedPU. This made results *worse*, not better – itself diagnostic, since a fix that makes things worse usually means a new bug was introduced, not that the underlying idea was wrong. Reading the raw per-episode training log (rather than only the final summary number) revealed the exact mechanism: the BC phase successfully drove policy entropy to near zero (a confident, deterministic clone of the constant policy, as intended). But the very next RL fine-tuning episode restarted the exploration-entropy bonus coefficient at its scheduled *maximum* value (0.05), with no awareness that the incoming policy was already a carefully-shaped, informative clone rather than a freshly-initialized random network. Fine-tuning was, in effect, actively erasing what BC had just taught it – entropy climbed back toward near-uniform ($H \approx 1.9$ of a maximum ≈ 1.95 for 7 actions) within roughly 100 episodes.

Fix (documented in code as the `entW_bc` comment inside `DeepA2C_Shared_v7.m`, Listing 16): the BC-phase entropy coefficient was softened to 0.03 (keeping the clone informative but not fully rigid, which also helps gradients flow during the subsequent fine-tune), and the post-BC fine-tuning entropy anneal was changed to *start* at 0.01 instead of resetting to 0.05 – fine-tuning must preserve the clone and only add corrections on top of it, not re-explore from scratch.

28.5 Root Cause 5 (Ruled Out, Not Found): Missing State Information

The Tier-A vs. Tier-B ablation (§31) shows no significant difference between the information-equal state and the richer state. This rules out "the agent just didn't have enough information" as an explanation for the campaign-wide null – Root Causes 1–4 above, not missing state, are the correct explanation.

29 Fix Iterations: `tier1`, `tier1b`, `tier1-nobc`

Three fix rounds were run at the headline point (`burst09dB`), each isolating a specific hypothesis about what was wrong.

Table 2: Fix Iterations at the Headline Point (PF Scheduler)

Run	Fixes Applied	A2C (Mbps)	vs. FixedPU	p	vs. OLLA
`burst09dB` (vanilla)	none	3.63	−0.7%	0.517	+10.4%***
`tier1`	PF-only training, BC (150 ep), grid $[-4, 2]$, actions ± 3	3.54	−5.0%	0.017*	+7.5%*
`tier1b`	`tier1` + actions $[-5, 3]$, BC entropy 0.03, post-BC anneal from 0.01	3.71	−0.4%	0.813 (n.s.)	+12.8%***
`tier1-nobc`	`tier1b` settings, **no** behavior cloning	3.54	−5.1%	0.060 (n.s.)	+7.5%**

`tier1` (Root Causes 1–3 addressed): still a loss, and a *larger* one on the point estimate than vanilla – the signal that motivated reading the raw training log and finding Root Cause 4.

`tier1b` (Root Cause 4 also addressed): reaches statistical parity with FixedPU ($p = 0.813$). First-transmission BLER matches almost exactly (0.439 vs. 0.443) – direct physical confirmation the policy converged to the *same operating point*, not merely the same aggregate throughput by coincidence. Critically, the 500 episodes of RL fine-tuning applied *on top of* the BC clone produced **zero** additional throughput over the clone itself – given a full training budget, richer state, and real gradient-based learning, the agent found nothing to add beyond imitating the oracle.

`tier1-nobc` (circularity control): `tier1b`'s number, while the "fixed" result, has an inherent limitation as evidence that RL doesn't need calibration data – it was initialized by directly imitating the calibrated comparator. `tier1-nobc` is identical to `tier1b` except `bc_warm_eps=0` – no cloning at all, learning purely from scratch. It reaches near-parity with FixedPU (-5.1%, not significant at $p = 0.060$) and a solid, significant win over OLLA ($+7.5\%$, $p = 0.005$) with zero access to the oracle's answers. `tier1b` demonstrates the achievable *ceiling*; `tier1-nobc` demonstrates that ceiling is *approximately reachable without the crutch.*

30 Final Performance Summary: All Schemes, Final (`tier1b`) Model

Table 3 is the single reference point for the project's bottom line – every metric collected by the harness, all seven schemes, headline point, PF scheduler, under the final `tier1b` model.

Table 3: Final Model Performance, Headline Point (PF Scheduler, $N = 10$, 5 seeds $\times$ 30 held-out traces)

Scheme	Cell Tp (Mbps)	Δ vs. Baseline	Edge Tp	BLER_1	Jain	vs. Deep A2C (Wilcoxon)
Baseline (no OLLA)	3.10 ± 0.25	(ref)	0.09	0.692	0.825	$+19.9\%$, $p = 1.1 \times 10^{-4}$***
OLLA (reactive)	3.29 ± 0.27	$+6.3\%$	0.13	0.640	0.845	$+12.8\%$, $p = 2.8 \times 10^{-4}$***
Fixed (best global constant)	3.77 ± 0.35	$+21.8\%$	0.17	0.444	0.884	-1.6%, $p = 0.262$ (n.s.)
FixedPU (oracle, per-class – pre-registered comparator)	3.73 ± 0.34	$+20.4\%$	0.17	0.443	0.881	$\mathbf{-0.4\%}$, $\mathbf{p = 0.813}$ **(n.s.)**
QL v7 (tabular)	3.12 ± 0.25	$+0.9\%$	0.10	0.690	0.833	$+19.4\%$, $p = 2.9 \times 10^{-6}$***
Tab AC v7 (tabular)	3.11 ± 0.21	$+0.4\%$	0.10	0.677	0.826	$+19.4\%$, $p = 2.9 \times 10^{-6}$***
Deep A2C v7 (`tier1b`, final)	$\mathbf{3.71 \pm 0.32}$	$\mathbf{+19.9\%}$	**0.17**	**0.439**	**0.879**	—

Read: the final model is statistically indistinguishable from the oracle-calibrated, per-user-class comparator ($p = 0.813$) – it neither wins nor loses the pre-registered test – while beating every reactive or tabular scheme by a wide, highly significant margin, and closely tracking FixedPU's edge-user throughput, BLER_1 operating point, and fairness index. This is the setpoint-effect finding in operational terms: the learned policy converges to essentially the same point on the throughput/reliability curve as the best possible fixed rule, not a distinguishable, state-adaptive one.

31 State Information Ablation (Tier A vs. Tier B)

Comparing the A2C-vs-FixedPU margin under the information-equal Tier-A state ($+0.1\%$, $p = 0.558$) against the richer Tier-B state (-0.7%, $p = 0.517$) at the same operating point shows no significant difference. Five explicit additional features – HARQ retransmission state, NACK-burst history, SINR slope, user speed, and observed burst indicator – buy nothing measurable. This directly rules out Root Cause 5 (§28): the campaign-wide null reflects an absence of exploitable state-conditional structure, not an information deficit in what the agent was shown.

32 Adversarial Review and the OLLA Target-BLER Sweep

32.1 The Objection

The final model's operating point runs a first-transmission BLER around 40–45%, far above OLLA's classical 10% design target. An independent adversarial review raised a specific, testable objection: is the entire measured advantage over OLLA simply because OLLA's target error rate was never re-tuned for the new HARQ physics (§5), which makes tolerating a higher first-attempt failure rate cheaper than it used to be? If moving OLLA's own target upward closes the gap, the "RL beats OLLA" finding would really be "someone forgot to re-tune a classical baseline for new physics" – a far less interesting result, and not really about learning at all.

32.2 Why the Test Was Cheap

OLLA's target error rate is a fixed algorithm parameter, not a learned quantity – it can simply be re-run at several target values on the same held-out traces already used throughout the campaign, and compared against the (already-trained, unmodified) neural network's fixed operating point. No retraining is required: minutes of computation, not hours, for a test that could have overturned a central project finding.

```
function sweep_olla_target_v7(targets)
% sweep_olla_target_v7 OLLA target-BLER re-tuning sweep, plus
% random-action and constant-offset controls, on the tier1b held-out traces.
    if nargin < 1 || isempty(targets), targets = [0.10 0.20 0.30 0.40 0.50]; end

    here = fileparts(mfilename('fullpath'));
    cd(here);
    addpath(fullfile(here, '..', 'v5'));
    addpath(fullfile(here, '..', 'case2_3'));
    if ~exist(fullfile(here,'logs'), 'dir'), mkdir(fullfile(here,'logs')); end
    diary(fullfile(here, 'logs', 'v7_olla_sweep_log.txt')); diary on;

    matf = fullfile(here, 'Dataset', 'results_v7_headline_tier1b.mat');
    if ~exist(matf, 'file'), error('missing %s', matf); end
    L = load(matf, 'results');
    env = L.results.env; env.scheduler = 'pf';
    A2Cs = L.results.policies.A2C;
    n_seeds = numel(A2Cs);

    simTime = 200; n_eval = 30; traceSeedBase = 9000;
    fprintf('\n===== OLLA target-BLER sweep (PF, %d traces, eval only) =====\n', n_eval);
    fprintf('actions=%s fixed=%+d class_tab=%s\n\n', ...
           mat2str(env.actions), env.fixed_offset, mat2str(env.fixed_class_tab));

    traces = struct('SINR',[],'obsS',[],'meta',[]); traces(n_eval).SINR = [];
    for j = 1:n_eval
        rng(traceSeedBase + j);
        [S, ~, mt] = generateChannel_v7(simTime, env);
        traces(j).SINR = S; traces(j).obsS = cqiFeedback_v7(S, env, mt.delays);
        traces(j).meta = mt;
    end

    scoreT = @(tp) sum(mean(tp(min(env.burn_in,size(tp,1)-1)+1:end, :), 1));
    scoreB = @(bl) mean(bl(~isnan(bl)));

    % OLLA frontier
    nT = numel(targets);
    tpO = zeros(nT, n_eval); blO = zeros(nT, n_eval);
    for q = 1:nT
        e = env; e.olla_target = targets(q);
        for j = 1:n_eval
            rng(88000 + 100*j + q);
            [tp, bl] = evalPolicy_v7('olla', [], traces(j).SINR, traces(j).obsS, e, traces(j).meta);
            tpO(q,j) = scoreT(tp); blO(q,j) = scoreB(bl);
        end
        fprintf(' OLLA target=%.2f : Tp = %.2f +/- %.2f Mbps BLER1 = %.3f\n', ...
               targets(q), mean(tpO(q,:)), 1.96*std(tpO(q,:))/sqrt(n_eval), mean(blO(q,:)));
    end

    % Controls: random action, constant -1
    tpR = zeros(1,n_eval); blR = zeros(1,n_eval);
    tpC = zeros(1,n_eval); blC = zeros(1,n_eval);
    eC = env; eC.fixed_offset = -1;
    for j = 1:n_eval
        rng(89000 + j);
        [tp, bl] = evalPolicy_v7('rand', [], traces(j).SINR, traces(j).obsS, env, traces(j).meta);
        tpR(j) = scoreT(tp); blR(j) = scoreB(bl);
        rng(90000 + j);
        [tp, bl] = evalPolicy_v7('fixed', [], traces(j).SINR, traces(j).obsS, eC, traces(j).meta);
        tpC(j) = scoreT(tp); blC(j) = scoreB(bl);
    end
    fprintf('\n rand-action : Tp = %.2f +/- %.2f Mbps BLER1 = %.3f\n', ...
           mean(tpR), 1.96*std(tpR)/sqrt(n_eval), mean(blR));
    fprintf(' const -1     : Tp = %.2f +/- %.2f Mbps BLER1 = %.3f\n', ...
           mean(tpC), 1.96*std(tpC)/sqrt(n_eval), mean(blC));

    % A2C reference (seed-averaged, same traces)
    tpA = zeros(1, n_eval); blA = zeros(1, n_eval);
    for j = 1:n_eval
```

```
        acc = 0; accb = 0;
        for s = 1:n_seeds
            rng(91000 + 100*j + s);
            [tp, bl] = evalPolicy_v7('a2c', A2Cs{s}, traces(j).SINR, traces(j).obsS, env, traces(j).meta);
            acc = acc + scoreT(tp); accb = accb + scoreB(bl);
        end
        tpA(j) = acc/n_seeds; blA(j) = accb/n_seeds;
    end
    fprintf(' Deep A2C     : Tp = %.2f +/- %.2f Mbps BLER1 = %.3f\n', ...
            mean(tpA), 1.96*std(tpA)/sqrt(n_eval), mean(blA));

    % Verdict: A2C vs best OLLA target (paired Wilcoxon)
    [~, qb] = max(mean(tp0, 2));
    fprintf('\n Best OLLA target = %.2f (Tp %.2f)\n', targets(qb), mean(tp0(qb,:)));
    if exist('signrank','file') == 2
        p = signrank(tpA, tp0(qb,:));
        mrg = 100*(mean(tpA) - mean(tp0(qb,:)))/mean(tp0(qb,:));
        fprintf(' A2C vs BEST-OLLA %+.1f%% p=%.4g\n', mrg, p);
        p2 = signrank(tpA, tpC);
        fprintf(' A2C vs const -1 %+.1f%% p=%.4g\n', ...
                100*(mean(tpA)-mean(tpC))/mean(tpC), p2);
        p3 = signrank(tpA, tpR);
        fprintf(' A2C vs rand     %+.1f%% p=%.4g\n', ...
                100*(mean(tpA)-mean(tpR))/mean(tpR), p3);
    end

    out = struct('targets',targets,'tp0',tp0,'bl0',bl0,'tpR',tpR,'tpC',tpC, ...
                 'tpA',tpA,'blA',blA,'blR',blR,'blC',blC,'n_eval',n_eval);
    save(fullfile(here,'Dataset','results_v7_olla_sweep.mat'), 'out', '-v7.3');
    fprintf('\n Saved: Dataset\\results_v7_olla_sweep.mat\n');
    diary off;
end
```

Listing 22: `sweep_olla_target_v7.m` — the decisive external-validation experiment: OLLA target-BLER re-tuning sweep plus random-action and action-set-centering controls, evaluated on the identical held-out traces used throughout the campaign.

32.3 Results

Table 4: OLLA Target-BLER Sweep and Controls (PF, 30 Held-Out Traces, No Retraining)

Configuration	Cell Tp (Mbps)	First-Tx BLER
OLLA, target=0.10 (classical)	3.32 ± 0.30	0.651
OLLA, target=0.20	3.29 ± 0.25	0.658
OLLA, target=0.30	3.24 ± 0.23	0.655
OLLA, target=0.40	3.31 ± 0.28	0.644
OLLA, target=0.50	3.15 ± 0.27	0.669
Random-action control	3.02 ± 0.28	0.721
Constant -1 control	3.61 ± 0.32	0.558
Deep A2C (final, `tier1b`)	$\mathbf{3.74 \pm 0.34}$	**0.427**

Verdict: re-tuning OLLA's target across a wide range does **not** close the gap – the best achievable OLLA throughput at any tested target (0.10, Tp= 3.32) remains well below Deep A2C (+12.9%, $p = 6.6 \times 10^{-4}$). The random-action control confirms A2C's behavior is clearly distinguishable from noise (+24.1% over random, $p = 2.4 \times 10^{-6}$). The single-constant control (-1, matched to A2C's action-range mean) is *not* significantly different from A2C (+3.7%, $p = 0.136$) – consistent with, not contradictory to, the setpoint-effect conclusion: A2C's advantage over OLLA specifically is real and survives the adversarial re-tuning test, but is well explained by a single scalar operating-point shift, exactly the same conclusion the FixedPU comparison (§30) supports independently.

33 Action Distribution: Direct Behavioral Evidence

Beyond matching throughput, does the learned policy's *behavior* look like a constant? `action_histogram_v7.m` re-evaluates `tier1b` on the 30 held-out traces, recording every scheduled decision's chosen OLLA-relative offset, and compares the resulting distribution against FixedPU's per-class target offsets. A policy whose actions cluster tightly on (or near) FixedPU's own class targets, rather than spreading meaningfully across the action set in a state-dependent way, is independent, non-throughput-based confirmation of the constant-policy interpretation – closing a logical gap the throughput-matching result alone cannot close (two very different policies could in principle average to the same throughput; this checks whether they are also behaviorally the same policy). See `fig12_action_distribution.png` in the figures folder and `v7_figure_explanations.pdf` for the rendered result and its detailed reading.

Part VIII Legacy Modules Appendix

Every function below predates v7 (most originate in the v5 line, one in `case2_3`) and is reused *unchanged*. Reusing audited, unchanged code for these low-level, cross-cutting utilities is a deliberate choice: it means any behavioral difference between schemes cannot be attributed to a subtle difference in, say, how ties are broken or how a transport block size is looked up – those are identical, by construction, everywhere they are used.

```
function TBS = tbsFromMCSandRB(mcs, rb)
% tbsFromMCSandRB TBS lookup for (MCS 0-14, RB 1-110).
    persistent tbsTable
    if isempty(tbsTable)
        tbsTable = createTBS_Table();
    end
    rb  = max(1, min(110, round(rb)));
    mcs = max(0, min(14, round(mcs)));
    TBS = tbsTable(mcs + 1, rb);
end
```

Listing 23: `tbsFromMCSandRB.m` (from `case2_3`) — transport block size lookup from a persistent table, indexed by MCS and RB count.

(See Listings 6–8, 12–14 above for `schedule_users_v5.m`, `schedContext_v5.m`, `sampleSched_v5.m`, `argmaxNoOp_v5.m`, `baseCQI_v5.m`, and `trendBin_v5.m` – reproduced in full at their point of first use in Parts 7 and III rather than duplicated here.)

Part IX Limitations and Conclusion

34 What This Investigation Does Not Establish

- A more extreme form of non-stationarity (bursts with dwell time shorter than the feedback delay itself, so a reactive loop genuinely cannot keep pace) was not tested and could change the finding.
- Untested state features beyond Tier B might carry signal Tier B does not capture – the ablation rules out *these five* features mattering, not that no feature could.
- The `tier1-nobc` near-parity gap, while not statistically significant, has not been resolved with a larger-power experiment designed specifically to distinguish "truly zero gap" from "small but real gap requiring more traces to detect."

These are natural next questions, not loose threads being concealed.

35 Conclusion

Across a redesigned environment specifically engineered to create the conditions under which state-conditional adaptivity should matter – costly error (D1), heterogeneous users (D2), abrupt regime changes (D3), realistic physics (D4), and a fair information comparison (D5) – a learned neural-network policy consistently reaches, but does not exceed, the performance of the best possible *unconditional* rule, once that rule is allowed to depend on which broad category of user it is talking to. It does, however, reliably and robustly outperform the classical reactive control algorithm actually deployed in real networks today, and that gain survives every attempt made to explain it away as a measurement artifact – calibration asymmetry, action-set clipping, undertraining, an entropy-collapse bug, a missing-information objection, and a mis-tuned-baseline objection were each tested directly and each, in turn, ruled out or fixed. The rigorously-obtained negative result against the oracle comparator, and the positive, well-supported result against the deployed classical comparator, are both defensible, evidenced findings from the same body of work – and the process by which each was established (pre-registration, code-level bug diagnosis over blind re-runs, deliberately adversarial review, and cheap decisive controls run before expensive ones) is intended to be as reusable as the numerical result itself.

Companion documents: `v7_beginners_guide.pdf` (first-principles narrative, no code), `v7_technical_process_report.pdf` (condensed reasoning chain), `v7_comprehensive_evaluation_report.pdf` (formal results and figures), `v7_figure_explanatior` (per-figure reading guide).